# Chapter 4 ▪ Objects and Classes

In this chapter, I

- Introduce you to object-oriented programming;
- Show you how you can create objects that belong to classes from the standard Java library; and
- Show you how to write your own classes.

If you do not have a background in object-oriented programming, you will want to read this chapter carefully. Object-oriented programming requires a different way of thinking than procedural languages. The transition is not always easy, but you do need some familiarity with object concepts to go further with Java.

For experienced C++ programmers, this chapter, like the previous chapter, presents familiar information; however, there are enough differences between the two languages that you should read the later sections of this chapter carefully. You'll find the C++ notes helpful for making the transition.

## 4.1. Introduction to Object-Oriented Programming

Object-oriented programming, or OOP for short, is the dominant programming paradigm these days, having replaced the "structured" or procedural programming techniques that were developed in the 1970s. Since Java is object-oriented, you have to be familiar with OOP to become productive with Java.

An object-oriented program is made of objects. Each object has a specific functionality, exposed to its users, and a hidden implementation. Many objects in your programs will be taken "off the shelf" from a library; others will be custom-designed. Whether you build an object or use a pre-built one might depend on your budget or time. But, basically, as long as an object satisfies your specifications, you don't care how the functionality is implemented.

Traditional structured programming consists of designing a set of procedures (or *algorithms*) to solve a problem. Once the procedures are determined, the traditional next step was to find appropriate ways to store the data. This is why the designer of the Pascal language, Niklaus Wirth, called his famous book on programming *Algorithms + Data Structures = Programs* (Prentice Hall, 1976). Notice that in

Wirth's title, algorithms come first, and data structures second. This reflects the way programmers worked at that time. First, they decided on the procedures for manipulating the data; then, they decided what structure to impose on the data to make the manipulations easier. OOP reverses the order: puts the data first, then looks at the algorithms to operate on the data.

For small problems, the breakdown into procedures works very well. But objects are more appropriate for larger problems. Consider a simple web browser. It might require 2,000 procedures for its implementation, all of which manipulate a set of global data. In the object-oriented style, there might be 100 classes with an average of 20 methods per class (see **Figure 4.1**). This structure is much easier for a programmer to grasp. It is also much easier to find bugs in. Suppose the data of a particular object is in an incorrect state. It is far easier to search for the culprit among the 20 methods that had access to that data item than among 2,000 procedures.

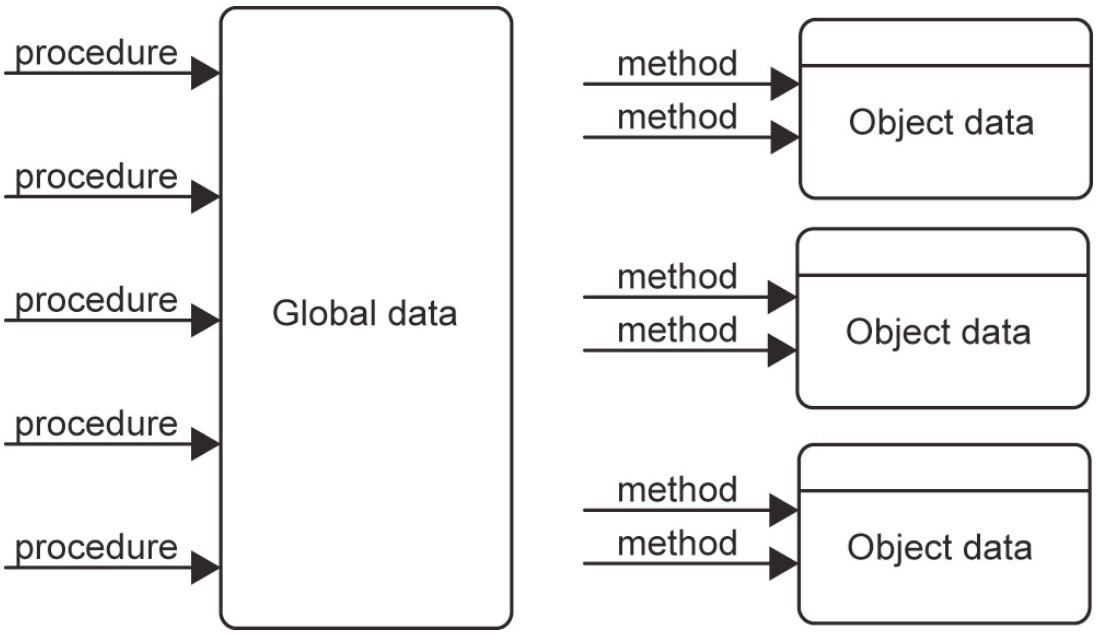



**Figure 4.1:** Procedural vs. OO programming

### 4.1.1. Classes

A *class* specifies how objects are made. Think of classes as cookie cutters; objects are the cookies themselves. When you *construct* an object from a class, you are said to have created an *instance* of the class.

As you have seen, all code that you write in Java is inside a class. The standard Java library supplies several thousand classes for such diverse purposes as user interface design, dates and calendars, and network programming. Nonetheless, in Java you still have to create your own classes to describe the objects of your application's problem domain.

*Encapsulation* (sometimes called *information hiding*) is a key concept in working with objects. Formally, encapsulation is simply combining data and behavior in one package and hiding the implementation details from the users of the object. The bits of data in an object are called its *instance fields*, and the procedures that operate on the data are called its *methods*. A specific object that is an instance of a class will have specific values of its instance fields. The set of those values is the current *state* of the object. Whenever you invoke a method on an object, its state may change.

The key to making encapsulation work is to have methods *never* directly access instance fields in a class other than their own. Programs should interact with object data *only* through the object's methods. Encapsulation is the way to give an object its "black box" behavior, which is the key to reuse and reliability. This means a class may totally change how it stores its data, but as long as it continues to use the same methods to manipulate the data, no other object will know or care.

When you start writing your own classes in Java, another tenet of OOP will make this easier: Classes can be built by *extending* other classes. Java, in fact, comes with a "cosmic superclass" called `Object`. All other classes extend this class. You will learn more about the `Object` class in **Chapter 5**.

When you extend an existing class, the new class has all the properties and methods of the class that you extend. You then supply new methods and instance fields that apply to your new class only. The concept of extending a class to obtain another class is called *inheritance*. See **Chapter 5** for more on inheritance.

### 4.1.2. Objects

To work with OOP, you should be able to identify three key characteristics of objects:

- The object's *behavior*—what can you do with this object, or what methods can you apply to it?
- The object's *state*—how does the object react when you invoke those methods?
- The object's *identity*—how is the object distinguished from others that may have the same behavior and state?

All objects that are instances of the same class share a family resemblance by supporting the same *behavior*. The behavior of an object is defined by the methods that you can call.

Next, each object stores information about what it currently looks like. This is the object's *state*. An object's state may change over time, but not spontaneously. A

change in the state of an object must be a consequence of method calls. (If an object's state changed without a method call on that object, someone broke encapsulation.)

However, the state of an object does not completely describe it, because each object has a distinct *identity*. For example, in an order processing system, two orders are distinct even if they request identical items. Notice that the individual objects that are instances of a class *always* differ in their identity and *usually* differ in their state.

These key characteristics can influence each other. For example, the state of an object can influence its behavior. (If an order is "shipped" or "paid," it may reject a method call that asks it to add or remove items. Conversely, if an order is "empty"—that is, no items have yet been ordered—it should not allow itself to be shipped.)

### 4.1.3. Identifying Classes

In a traditional procedural program, you start the process at the top, with the `main` function. When designing an object-oriented system, there is no "top," and newcomers to OOP often wonder where to begin. The answer is: Identify your classes and then add methods to each class.

A simple rule of thumb in identifying classes is to look for nouns in the problem analysis. Methods, on the other hand, correspond to verbs.

For example, in an order-processing system, some of the nouns are

- Item
- Order
- Shipping address
- Payment
- Account

These nouns may lead to the classes `Item`, `Order`, and so on.

Next, look for verbs. Items are *added* to orders. Orders are *shipped* or *canceled*. Payments are *applied* to orders. With each verb, such as "add," "ship," "cancel," or "apply," you identify the object that has the major responsibility for carrying it out. For example, when a new item is added to an order, the order object should be the one in charge because it knows how it stores and sorts items. That is, `add` should be a method of the `Order` class that has an `Item` object as a parameter.

Of course, the "noun and verb" is but a rule of thumb; only experience can help you decide which nouns and verbs are the important ones when building your classes.

### 4.1.4. Relationships between Classes

The most common relationships between classes are

- *Dependence* ("uses–a")
- *Aggregation* ("has–a")
- *Inheritance* ("is–a")

The *dependence*, or "uses–a" relationship, is the most obvious and also the most general. For example, the `Order` class uses the `Account` class because `Order` objects need to access `Account` objects to check for credit status. But the `Item` class does not depend on the `Account` class, because `Item` objects never need to worry about customer accounts. Thus, a class depends on another class if its methods use or manipulate objects of that class.

Try to minimize the number of classes that depend on each other. The point is, if a class `A` is unaware of the existence of a class `B`, it is also unconcerned about any changes to `B`. (And this means that changes to `B` do not introduce bugs into `A`.) In software engineering terminology, you want to minimize the *coupling* between classes.

The *aggregation*, or "has–a" relationship, is easy to understand because it is concrete; for example, an `Order` object contains `Item` objects. Containment means that objects of class `A` contain objects of class `B`.



**Note:** Some methodologists view the concept of aggregation with disdain and prefer to use a more general "association" relationship. From the point of view of modeling, that is understandable. But for programmers, the "has–a" relationship makes a lot of sense. I like to use aggregation for another reason as well: The standard notation for associations is less clear. See **Table 4.1**.

The *inheritance*, or "is–a" relationship, expresses a relationship between a more special and a more general class. For example, a `RushOrder` class inherits from an `Order` class. The specialized `RushOrder` class has special methods for priority handling and a different method for computing shipping charges, but its other methods, such as adding items and billing, are inherited from the `Order` class. In general, if class `D` extends class `C`, class `D` inherits methods from class `C` but has more capabilities. (See **Chapter 5** which discusses this important notion at some length.)

Many programmers use the UML (Unified Modeling Language) notation to draw *class diagrams* that describe the relationships between classes. You can see an example of such a diagram in **Figure 4.2**. You draw classes as rectangles, and relationships as arrows with various adornments. **Table 4.1** shows the UML arrow styles that this book uses.

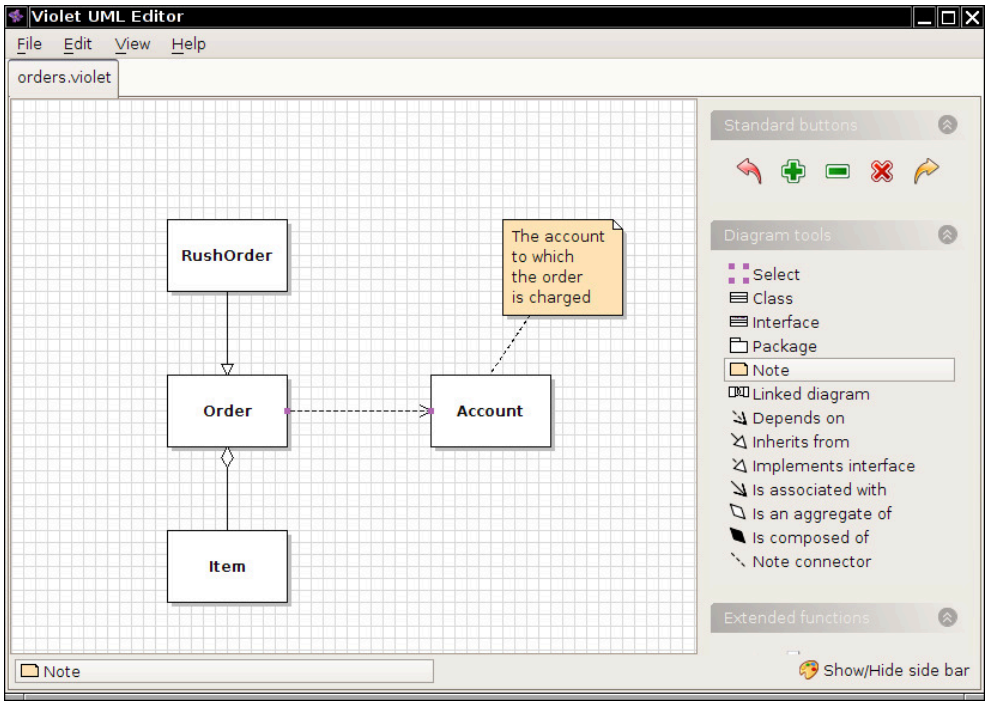



**Figure 4.2:** A class diagram

**Table 4.1:** UML Notation for Class Relationships

| Relationship | UML Connector |
|---|---|
| Inheritance | |
| Interface implementation | |
| Dependency | |
| Aggregation | |

## 4.2. Using Predefined Classes

You can't do anything in Java without classes, and you have already seen several classes at work. However, not all of these show off the typical features of object orientation. Take, for example, the `Math` class. You have seen that you can use methods of the `Math` class, such as `Math.random`, without needing to know how they are implemented—all you need to know is the name and parameter types (if any). That's the point of encapsulation, and it will certainly be true of all classes. But the `Math` class *only* encapsulates functionality; it neither needs nor hides data. Since there is no data, you do not need to worry about making objects and initializing their instance fields—there aren't any!

In the next section, we will look at a more typical class, the `Date` class. You will see how to construct objects and call methods of this class.

### 4.2.1. Objects and Object Variables

To work with objects, you first construct them and specify their initial state. Then you apply methods to the objects.

In the Java programming language, you use *constructors* to construct new instances. A constructor is a special method whose purpose is to construct and initialize objects. Let us look at an example. The standard Java library contains a `Date` class. Its objects describe points in time, such as December 31, 1999, 23:59:59 GMT.

---



**Note:** You may be wondering: Why use a class to represent dates rather than (as in some languages) a built-in type? For example, Visual Basic has a built-in date type, and programmers can specify dates in the format `#12/31/1999#`. On the surface, this sounds convenient—programmers can simply use the built-in date type without worrying about classes. But actually, how suitable is the Visual Basic design? In some locales, dates are specified as month/day/year, in others as day/month/year. Are the language designers really equipped to foresee these kinds of issues? If they do a poor job, the language becomes an unpleasant muddle, but unhappy programmers are powerless to do anything about it. With classes, the design task is offloaded to a library designer. If the class is not perfect, other programmers can easily write their own classes to enhance or replace the system classes. (To prove the point: The Java date library started out a bit muddled, and it has been redesigned twice.)

---

Constructors always have the same name as the class name. Thus, the constructor for the `Date` class is called `Date`. To construct a `Date` object, combine the constructor with the `new` operator, as follows:

```
new Date()
```

This expression constructs a new object. The object is initialized to the current date and time.

If you like, you can pass the object to a method:

```
IO.println(new Date());
```

Alternatively, you can apply a method to the object that you just constructed. One of the methods of the `Date` class is the `toString` method. That method yields a string representation of the date. Here is how you would apply the `toString` method to a newly constructed `Date` object:

```
String s = new Date().toString();
```

In these two examples, the constructed object is used only once. Usually, you will want to hang on to the objects that you construct so that you can keep using them. Simply store the object in a variable:

```
Date rightNow = new Date();
```

**Figure 4.3** shows the object variable `rightNow` that refers to the newly constructed object.

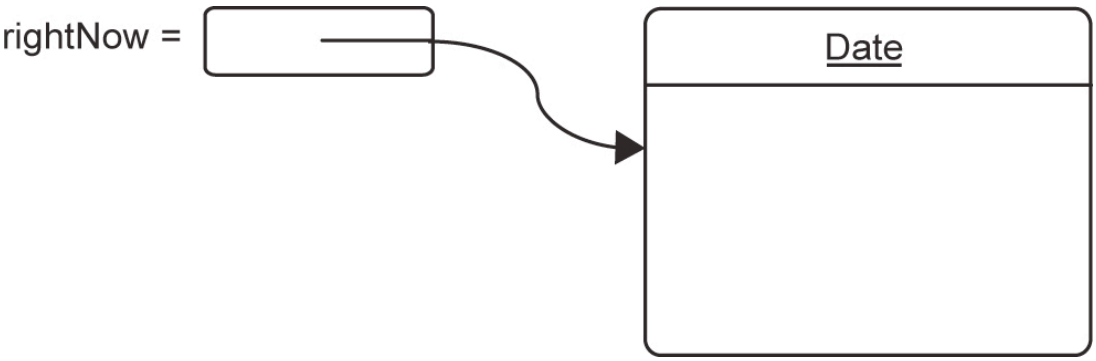



**Figure 4.3:** Creating a new object

There is an important difference between objects and object variables. For example, the statement

```
Date startTime; // startTime doesn't refer to any object
```

defines an object variable, `startTime`, that can refer to objects of type `Date`. It is important to realize that the variable `startTime` *is not an object* and, in fact, does not even refer to an object yet. You cannot use any `Date` methods on this variable at this time. The statement

```
s = startTime.toString(); // not yet
```

would cause a compile-time error.

You must first initialize the `startTime` variable. You have two choices. Of course, you can initialize the variable so that it refers to a newly constructed object:

```
startTime = new Date();
```

Or you can set the variable to refer to an existing object:

```
startTime = rightNow;
```

Now both variables refer to the *same* object (see **Figure 4.4**).

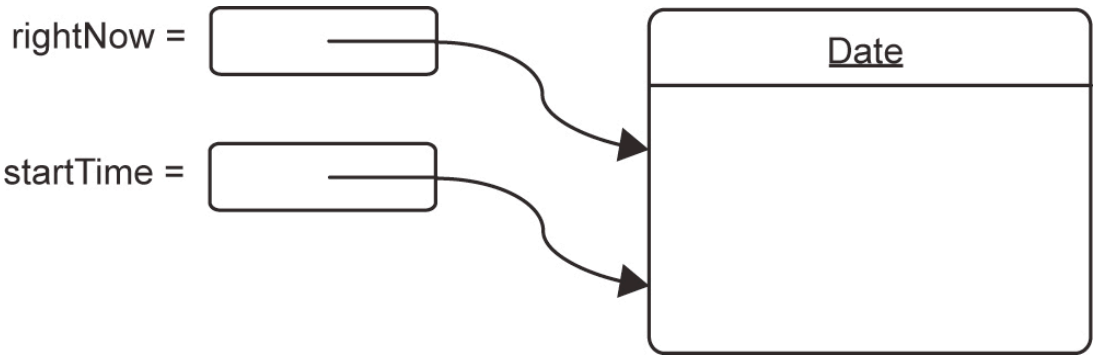



**Figure 4.4:** Object variables that refer to the same object

It is important to realize that an object variable doesn't actually contain an object. It only *refers* to an object.

In Java, the value of any object variable is a reference to an object that is stored elsewhere. The return value of the `new` operator is also a reference. A statement such as

```
Date startTime = new Date();
```

has two parts. The expression `new Date()` makes an object of type `Date`, and its value is a reference to that newly created object. That reference is then stored in the `startTime` variable.

You can explicitly set an object variable to `null` to indicate that it currently refers to no object.

```
startTime = null;
. . .
if (startTime != null)
    IO.println(startTime);
```

I discuss `null` in more detail in **Section 4.3.5**.



**Note:** If you are a C++ programmer, do not confuse C++ reference variables with Java references. In C++ there are no null references, and references cannot be assigned. You should think of Java object variables as analogous to *object pointers* in C++. For example,

```
Date rightNow; // Java
```

is equivalent to

```
Date* rightNow; // C++ pointer
```

and not

```
Date& rightNow; // C++ reference
```

In C++, pointers make you nervous because they are so error-prone. It is easy to create bad pointers or to mess up memory management. In Java, these problems simply go away. If you use an uninitialized pointer, the run-time system will reliably generate a runtime error instead of producing random results. You don't have to worry about memory management, because the garbage collector takes care of it.

### 4.2.2. The `LocalDate` Class of the Java Library

In the preceding examples, we used the `Date` class that is a part of the standard Java library. An instance of the `Date` class has a state—namely, *a particular point in time*.

Although you don't need to know this when you use the `Date` class, the time is represented by the number of milliseconds (positive or negative) from a fixed point, the so-called *epoch*, which is 00:00:00 UTC, January 1, 1970. UTC is the Coordinated Universal Time, the scientific time standard which is, for practical purposes, the same as the more familiar GMT, or Greenwich Mean Time.

But as it turns out, the `Date` class is not very useful for manipulating the kind of calendar information that humans use for dates, such as "December 31, 1999." This particular description of a day follows the Gregorian calendar, which is the calendar used in most countries of the world. The same point in time would be described quite differently in the Chinese or Hebrew lunar calendars, not to mention the calendar used by your customers from Mars.

---



**Note:** Throughout human history, civilizations grappled with the design of calendars to attach names to dates and bring order to the solar and lunar cycles. For a fascinating explanation of calendars around the world, from the French Revolutionary calendar to the Mayan long count, see *Calendrical Calculations* by Nachum Dershowitz and Edward M. Reingold (Cambridge University Press, 4th ed., 2018).

---

The library designers decided to separate the concerns of keeping time and attaching names to points in time. Therefore, the standard Java library contains two separate classes: the `Date` class, which represents a point in time, and the `LocalDate` class, which expresses days in the familiar calendar notation. Nowadays there is a much more robust set of classes for manipulating various aspects of date and time—see **Chapter 6 of Volume II**.

Separating time measurement from calendars is good object-oriented design. In general, it is a good idea to use different classes to express different concepts.

You do not use a constructor to construct objects of the `LocalDate` class. Instead, use static *factory methods* that call constructors on your behalf. The expression

```
LocalDate.now()
```

constructs a new object that represents the date at which the object was constructed.

You can construct an object for a specific date by supplying year, month, and day:

```
LocalDate.of(1999, 12, 31)
```

Of course, you will usually want to store the constructed object in an object variable:

```
LocalDate newYearsEve = LocalDate.of(1999, 12, 31);
```

Once you have a `LocalDate` object, you can find out the year, month, and day with the methods `getYear`, `getMonthValue`, and `getDayOfMonth`:

```
int year = newYearsEve.getYear(); // 1999
int month = newYearsEve.getMonthValue(); // 12
int day = newYearsEve.getDayOfMonth(); // 31
```

This may seem pointless because they are the very same values that you just used to construct the object. But sometimes, you have a date that has been computed, and then you will want to invoke those methods to find out more about it. For example, the `plusDays` method yields a new `LocalDate` that is a given number of days away from the object to which you apply it:

```
LocalDate aThousandDaysLater = newYearsEve.plusDays(1000);
year = aThousandDaysLater.getYear(); // 2002
month = aThousandDaysLater.getMonthValue(); // 09
day = aThousandDaysLater.getDayOfMonth(); // 26
```

The `LocalDate` class has encapsulated instance fields to maintain the date to which it is set. Without looking at the source code, it is impossible to know the representation that the class uses internally. But, of course, the point of encapsulation is that this doesn't matter. What matters are the methods that a class exposes.

---



**Note:** Actually, the `Date` class also has methods to get the day, month, and year, called `getDay`, `getMonth`, and `getYear`, but these methods are *deprecated*. A method is deprecated when a library designer realizes that the method should have never been introduced in the first place.

These methods were a part of the `Date` class before the library designers realized that it makes more sense to supply separate classes to deal with calendars. When an earlier set of calendar classes was introduced in Java 1.1, the `Date` methods were tagged as deprecated. You can still use them in your programs, but you will get unsightly compiler warnings if you do. It is a good idea to stay away from using deprecated methods because they may be removed in a future version of the library.

---



**Tip:** The JDK provides the `jdeprscan` tool for checking whether your code uses deprecated features of the Java API. See https://docs.oracle.com/en/java/javase/25/docs/specs/man/jdeprscan.html for instructions.

### 4.2.3. Mutator and Accessor Methods

Have another look at the `plusDays` method call that you saw in the preceding section:

```
LocalDate aThousandDaysLater = newYearsEve.plusDays(1000);
```

What happens to `newYearsEve` after the call? Has it been changed to be a thousand days later? As it turns out, it has not. The `plusDays` method yields a new `LocalDate` object, which is then assigned to the `aThousandDaysLater` variable. The original object remains unchanged. We say that the `plusDays` method does not *mutate* the object on which it is invoked. (This is similar to the `toUpperCase` method of the `String` class that you saw in **Chapter 3**. When you call `toUpperCase` on a string, that string stays the same, and a new string with uppercase characters is returned.)

An earlier version of the Java library had a different class for dealing with calendars, called `GregorianCalendar`. This code snippet adds a thousand days to a date represented by that class:

```
GregorianCalendar someDay = new GregorianCalendar(1999, 11, 31);
    // odd feature of that class: month numbers go from 0 to 11
someDay.add(Calendar.DAY_OF_MONTH, 1000);
```

Unlike the `LocalDate.plusDays` method, the `GregorianCalendar.add` method is a *mutator method*. After invoking it, the state of the `someDay` object has changed. Here is how you can find out the new state:

```
year = someDay.get(Calendar.YEAR); // 2002
month = someDay.get(Calendar.MONTH) + 1; // 09
day = someDay.get(Calendar.DAY_OF_MONTH); // 26
```

That's why the variable is called `someDay` and not `newYearsEve`—it is no longer new year's eve after calling the mutator method.

In contrast, methods that only access objects without modifying them are sometimes called *accessor methods*. For example, `LocalDate.getYear` and

`GregorianCalendar.get` are accessor methods.

---



**Note:** In some object-oriented languages, you can explicitly declare accessor or mutator methods. For example, in C++, the `const` suffix denotes accessor methods. However, in the Java programming language, no special syntax distinguishes accessors from mutators.

---

I finish this section with a program that puts the `LocalDate` class to work. The program displays a calendar for the current month, like this:

```
Mon Tue Wed Thu Fri Sat Sun
                        1
  2   3   4   5   6   7   8
  9  10  11  12  13  14  15
 16  17  18  19  20  21  22
 23  24  25  26* 27  28  29
 30
```

The current day is marked with an asterisk (`*`). As you can see, the program needs to know how to compute the length of a month and the weekday of a given day.

Let us go through the key steps of the program. First, we construct an object that is initialized with the current date.

```
LocalDate date = LocalDate.now();
```

We capture the current month and day.

```
int month = date.getMonthValue();
int today = date.getDayOfMonth();
```

Then we set `date` to the first of the month and get the weekday of that date.

```
date = date.minusDays(today - 1); // set to start of month
DayOfWeek weekday = date.getDayOfWeek();
int value = weekday.getValue(); // 1 = Monday, . . . , 7 = Sunday
```

The variable `weekday` is set to an object of type `DayOfWeek`. We call the `getValue` method of that object to get a numerical value for the weekday. This yields an integer that follows the international convention where the weekend

comes at the end of the week, returning `1` for Monday, `2` for Tuesday, and so on. Sunday has the value `7`.

Note that the first line of the calendar is indented, so that the first day of the month falls on the appropriate weekday. Here is the code to print the header and the indentation for the first line:

```
IO.println("Mon Tue Wed Thu Fri Sat Sun");
for (int i = 1; i < value; i++) {
    IO.print("    ");
}
```

Now, we are ready to print the body of the calendar. We enter a loop in which `date` traverses the days of the month.

In each iteration, we print the date value. If `date` is today, the date is marked with an `*`. Then, we advance `date` to the next day. When we reach the beginning of each new week, we print a new line:

```
while (date.getMonthValue() == month) {
    IO.print("%3d".formatted(date.getDayOfMonth()));
    if (date.getDayOfMonth() == today) {
        IO.print("*");
    }
    else {
        IO.print(" ");
    }
    date = date.plusDays(1);
    if (date.getDayOfWeek().getValue() == 1) {
        IO.println();
    }
}
```

When do we stop? We don't know whether the month has 31, 30, 29, or 28 days. Instead, we keep iterating while `date` is still in the current month.

**Listing 4.1** shows the complete program.

As you can see, the `LocalDate` class makes it possible to write a calendar program that takes care of complexities such as weekdays and the varying month lengths. You don't need to know *how* the `LocalDate` class computes months and weekdays. You just use the *interface* of the class—the methods such as `plusDays` and `getDayOfWeek`.

The point of this example program is to show you how you can use the interface of a class to carry out fairly sophisticated tasks without having to know the implementation details.

**Listing 4.1 CalendarDemo.java**

```
/**
 * This program prints a calendar for the current month.
 */
void main() {
    LocalDate date = LocalDate.now();
    int month = date.getMonthValue();
    int today = date.getDayOfMonth();

    date = date.minusDays(today - 1); // set to start of month
    DayOfWeek weekday = date.getDayOfWeek();
    int value = weekday.getValue(); // 1 = Monday, . . . , 7 = Sunday

    IO.println("Mon Tue Wed Thu Fri Sat Sun");
    for (int i = 1; i < value; i++) {
        IO.print("    ");
    }
    while (date.getMonthValue() == month) {
        IO.print("%3d".formatted(date.getDayOfMonth()));
        if (date.getDayOfMonth() == today) {
            IO.print("*");
        }
        else {
            IO.print(" ");
        }
        date = date.plusDays(1);
        if (date.getDayOfWeek().getValue() == 1) {
            IO.println();
        }
    }
    if (date.getDayOfWeek().getValue() != 1) {
        IO.println();
    }
}
```

`java.time.LocalDate` 8

- `static LocalDate now()`
  constructs an object that represents the current date.
- `static LocalDate of(int year, int month, int day)`
  constructs an object that represents the given date.

- `int getYear()`
- `int getMonthValue()`
- `int getDayOfMonth()`
  get the year, month, and day of this date.
- `DayOfWeek getDayOfWeek()`
  gets the weekday of this date as an instance of the `DayOfWeek` class. Call `getValue` on the `DayOfWeek` instance to get a weekday between `1` (Monday) and `7` (Sunday).
- `LocalDate plusDays(int n)`
- `LocalDate minusDays(int n)`
  yield the date that is `n` days after or before this date.

## 4.3. Defining Your Own Classes

In **Chapter 3**, you started writing simple programs with compact source files, each with a `main` method. Now the time has come to show you how to write the kind of "workhorse classes" that are needed for more sophisticated applications. These classes typically do not have a `main` method. Instead, they have their own instance fields and methods. To build a complete program, you combine several source files, one of which has a `main` method.

### 4.3.1. An `Employee` Class

The simplest form for a class definition in Java is

```
class ClassName {
    field1
    field2
    . . .
    constructor1
    constructor2
    . . .
    method1
    method2
    . . .
}
```

Consider the following, very simplified, version of an `Employee` class that might be used by a business in writing a payroll system:

```
class Employee {
    // instance fields
    private String name;
    private double salary;
```

```
    private LocalDate hireDay;

    // constructor
    Employee(String n, double s, int year, int month, int day) {
        name = n;
        salary = s;
        hireDay = LocalDate.of(year, month, day);
    }

    // a method
    String getName() {
        return name;
    }

    // more methods
    . . .
}
```

We break down the implementation of this class, in some detail, in the sections that follow. First, though, the `main` method in **Listing 4.2** shows the `Employee` class in action.

In the program, we construct an `Employee` array and fill it with three `Employee` objects. Next, we use the `raiseSalary` method of the `Employee` class to raise each employee's salary by 5%. Finally, we print out information about each employee, by calling the `getName`, `getSalary`, and `getHireDay` methods.

```
for (Employee e : staff) {
    IO.println("name=" + e.getName()
        + ",salary=" + e.getSalary()
        + ",hireDay=" + e.getHireDay());
}
```

**Listing 4.2 EmployeeDemo.java**

```
1  /**
2   * This program tests the Employee class.
3   */
4  void main() {
5      // fill the staff array with three Employee objects
6      Employee[] staff = new Employee[3];
7
8      staff[0] = new Employee("Carl Cracker", 75000, 1987, 12, 15);
9      staff[1] = new Employee("Harry Hacker", 50000, 1989, 10, 1);
10     staff[2] = new Employee("Tony Tester", 40000, 1990, 3, 15);
11
```

```
12      // raise everyone's salary by 5%
13      for (Employee e : staff) {
14          e.raiseSalary(5);
15      }
16
17      // print out information about all Employee objects
18      for (Employee e : staff) {
19          IO.println("name=" + e.getName() + ",salary=" + e.getSalary()
20                  + ",hireDay=" + e.getHireDay());
21      }
22  }
23
24  /**
25   * Variations of this class will be used throughout the book.
26   */
27  class Employee {
28      private String name;
29      private double salary;
30      private LocalDate hireDay;
31
32      Employee(String n, double s, int year, int month, int day) {
33          name = n;
34          salary = s;
35          hireDay = LocalDate.of(year, month, day);
36      }
37
38      String getName() {
39          return name;
40      }
41
42      double getSalary() {
43          return salary;
44      }
45
46      LocalDate getHireDay() {
47          return hireDay;
48      }
49
50      void raiseSalary(double byPercent) {
51          double raise = salary * byPercent / 100;
52          salary += raise;
53      }
54  }
```

### 4.3.2. Dissecting the `Employee` Class

In the sections that follow, we will dissect the `Employee` class. Let's start with the methods in this class. As you can see by examining the source code, this class has one constructor and four methods:

```
Employee(String n, double s, int year, int month, int day)
String getName()
double getSalary()
LocalDate getHireDay()
void raiseSalary(double byPercent)
```

In many ways, this is a great example of a class. It describes an entity of a business domain. There is state: the name and salary. The state can change, on the happy occasion of a raise. There are different kinds of employees: managers, executives, and so on, which can be modeled as a hierarchy of related classes.

But the example is not perfect. Should the name be a string, or should there be a separate `Name` class, so one can properly deal with the finer points of names, such as first and last names and middle initials? Is `double` the right type for a salary? Why pass year, month, and day separately to the constructor instead of a `LocalDate` object? Would it be better to have immutable state? And is it really a good idea to use separate classes for managers and executives, as we will do in the next chapter?

These are valid concerns, but I decided to stick with this `Employee` class. It models a familiar context, and it is convenient for exploring Java features by considering various implementations.

---



**Note:** It is common to declare classes and methods as `public`. We discuss access levels later in this chapter. For our first programs, the default access rules are sufficient.

---

Notice the three instance fields that will hold the data manipulated inside an instance of the `Employee` class.

```
private String name;
private double salary;
private LocalDate hireDay;
```

The `private` keyword makes sure that the *only* methods that can access these instance fields are the methods of the `Employee` class itself. No outside method can read or write to these fields.

---



**Note:** You could omit the `private` keyword with your instance fields, but it would be a very bad idea. Having non-private instance fields would allow other parts of the program to read and modify the instance fields, completely ruining encapsulation. If it is possible to modify fields, in my experi-

ence, some code *will* take advantage of that access privilege when you least expect it. It is best to make all your instance fields `private`.

---

Finally, note that two of the instance fields are themselves objects: The `name` and `hireDay` fields are references to `String` and `LocalDate` objects. This is quite usual: Classes will often contain instance fields of class type.

### 4.3.3. First Steps with Constructors

Let's look at the constructor listed in our `Employee` class.

```
Employee(String n, double s, int year, int month, int day) {
    name = n;
    salary = s;
    hireDay = LocalDate.of(year, month, day);
}
```

As you can see, the name of the constructor is the same as the name of the class. This constructor runs when you construct objects of the `Employee` class—giving the instance fields the initial state you want them to have.

For example, when you create an instance of the `Employee` class with code like this:

```
new Employee("James Bond", 100000, 1950, 1, 1)
```

you have set the instance fields as follows:

```
name = "James Bond";
salary = 100000;
hireDay = LocalDate.of(1950, 1, 1); // January 1, 1950
```

There is an important difference between constructors and other methods. A constructor can only be called in conjunction with the `new` operator. You can't apply a constructor to an existing object to reset the instance fields. For example,

```
james.Employee("James Bond", 250000, 1950, 1, 1) // ERROR
```

is a compile-time error.

We will have more to say about constructors later in this chapter. For now, keep the following in mind:

- A constructor has the same name as the class.
- A class can have more than one constructor.
- A constructor can have zero, one, or more parameters.
- A constructor has no return value.
- A constructor is always called with the `new` operator.



**Note:** In Python, you don't use the `new` operator with constructors. Still, all Python objects are allocated on the heap. In C++, you can construct objects on the stack (without `new`) or on the heap (with `new`).

```
Employee number007("James Bond", 100000, 1950, 1, 1); // C++, on stack
Employee* mata_hari = new Employee("Mata Hari", 100000, 1876, 8, 7); // C++, o
```



**Caution:** Be careful not to introduce local variables with the same names as the instance fields. For example, the following constructor will not set the `name` or `salary` instance fields:

```
Employee(String n, double s, . . .) {
    String name = n; // ERROR
    double salary = s; // ERROR
    . . .
}
```

The constructor declares *local* variables `name` and `salary`. These variables are only accessible inside the constructor. They *shadow* the instance fields with the same name. Some programmers accidentally write this kind of code when they type faster than they think, because their fingers are used to adding the data type. This is a nasty error that can be hard to track down. You just have to be careful in all of your methods to not use variable names that equal the names of instance fields.

### 4.3.4. Declaring Local Variables with `var`

You can declare local variables with the `var` keyword instead of specifying their type, provided their type can be inferred from the initial value. For example, instead of declaring

```
Employee harry = new Employee("Harry Hacker", 50000, 1989, 10, 1);
```

you simply write

```
var harry = new Employee("Harry Hacker", 50000, 1989, 10, 1);
```

This is nice since it avoids the repetition of the type name `Employee`.

From now on, I will use the `var` notation in those cases where the type is obvious from the right-hand side without any knowledge of the Java API. But I won't use `var` with numeric types such as `int`, `long`, or `double` so that you don't have to look out for the difference between `0`, `0L`, and `0.0`. Once you are more experienced with the Java API, you may want to use the `var` keyword more frequently.

Note that the `var` keyword can only be used with *local* variables inside methods. You must always declare the types of parameters and fields.



**Note:** Many programmers use the `var` keyword quite liberally, whenever they think that the variable type is obvious or obviously uninteresting. A good set of guidelines is at **https://openjdk.org/projects/amber/guides/lvti-style-guide**.

### 4.3.5. Working with `null` References

In **Section 4.2.1**, you saw that an object variable holds a reference to an object, or the special value `null` to indicate the absence of an object.

This sounds like a convenient mechanism for dealing with special situations, such as an unknown name or hire date. But you need to be very careful with `null` values.

If you apply a method to a `null` value, a `NullPointerException` occurs.

```
LocalDate rightNow = null;
String s = rightNow.toString(); // NullPointerException
```

This is a serious error, similar to an "index out of bounds" exception. If your program does not "catch" an exception, it is terminated. Normally, programs don't catch these kinds of exceptions but rely on programmers not to cause them in the first place.



**Tip:** When your program is terminated with a `NullPointerException`, the stack trace shows you in which line of your code the problem occurred. Since Java 14, the error message includes the name of the variable or method with the `null` value. For example, in a call

```
String s = e.getHireDay().toString();
```

the error message tells you whether `e` was `null` or `getHireDay` returned `null`.

---

When you define a class, it is a good idea to be clear about which fields can be `null`. In our example, we don't want the `name` or `hireDay` field to be `null`. (We don't have to worry about the `salary` field. It has primitive type and can never be `null`.)

The `hireDay` field is guaranteed to be non-`null` because it is initialized with a new `LocalDate` object. But `name` will be `null` if the constructor is called with a `null` argument for `n`.

There are two solutions. The "permissive" approach is to turn a `null` argument into an appropriate non-`null` value:

```
if (n == null) name = "unknown"; else name = n;
```

The `Objects` class has a convenience method for this purpose:

```
Employee(String n, double s, int year, int month, int day) {
    name = Objects.requireNonNullElse(n, "unknown");
    . . .
}
```

The "tough love" approach is to reject a `null` argument:

```
Employee(String n, double s, int year, int month, int day) {
    name = Objects.requireNonNull(n, "The name cannot be null");
    . . .
}
```

If someone constructs an `Employee` object with a `null` name, then a `NullPointerException` occurs. At first glance, that may not seem a useful remedy. But there are two advantages:

1. The exception report has a description of the problem.
2. The exception report pinpoints the location of the problem. Otherwise, a `NullPointerException` would have occurred elsewhere, with no easy way of tracing it back to the faulty constructor argument.



**Note:** Whenever a parameter is an object reference, ask yourself whether you really intend to model values that can be present or absent. If not, the "tough love" approach of throwing an exception is preferred.

### 4.3.6. Implicit and Explicit Parameters

Methods operate on objects and access their instance fields. For example, the method

```
void raiseSalary(double byPercent) {
    double raise = salary * byPercent / 100;
    salary += raise;
}
```

sets a new value for the `salary` instance field in the object on which this method is invoked. Consider the call

```
number007.raiseSalary(5);
```

The effect is to increase the value of the `number007.salary` field by 5%. More specifically, the call executes the following instructions:

```
double raise = number007.salary * 5 / 100;
number007.salary += raise;
```

The `raiseSalary` method is called with two arguments. The first argument, called the *implicit* argument, is the object of type `Employee` that appears before the method name. The second argument, the number inside the parentheses after the method name, is an *explicit* argument.

The method declaration has a parameter variable for the explicit argument, namely `double byPercent`. However, no parameter variable is declared for the implicit argument.

Every method has an *implicit parameter*, whose name is the keyword `this`, which is initialized with the implicit argument. If you like, you can write the `raiseSalary` method as follows:

```
void raiseSalary(double byPercent) {
    double raise = this.salary * byPercent / 100;
    this.salary += raise;
}
```

Some programmers prefer that style because it clearly distinguishes between instance fields and local variables.

---



**Note:** I find the "implicit/explicit" terminology useful for describing the behavior of a method, but it is not an official part of Java. The Java Language Specification simply talks about "the object for which the method was invoked."

When object-oriented programming was first invented, objects were said to communicate by sending messages to each other. In that terminology, the object to which the message is sent is called the *receiver*. In Java, sending a message to a receiver is the same as invoking a method on an object.

---

### 4.3.7. Benefits of Encapsulation

Finally, let's look more closely at the rather simple `getName`, `getSalary`, and `getHireDay` methods.

```
String getName() {
    return name;
}

double getSalary() {
    return salary;
}

LocalDate getHireDay() {
    return hireDay;
}
```

These are obvious examples of accessor methods. As they simply return the values of instance fields, they are sometimes called *field accessors*.

Wouldn't it be easier to make the `name`, `salary`, and `hireDay` fields directly accessible, instead of having separate accessor methods?

Using methods gives you more control and safety. Consider the `salary` field which is only changed by the `raiseSalary` method. Should the value ever turn out wrong, only that method needs to be debugged. Had the `salary` field been directly accessible, the culprit for messing up the value could have been anywhere.

Sometimes, it happens that you want to get and set the value of an instance field. Then you need to supply *three* items:

- A private instance field;
- A field accessor method; and
- A field mutator method.

This is a lot more tedious than making the instance field directly accessible, but there are considerable benefits.

First, you can change the internal implementation without affecting any code other than the methods of the class. For example, if the storage of the name is changed to

```
private String firstName;
private String lastName;
```

then the `getName` method can be changed to return

```
firstName + " " + lastName
```

This change is completely invisible to the remainder of the program.

Of course, the accessor and mutator methods may need to do a lot of work to convert between the old and the new data representation. That leads us to our second benefit: Mutator methods can perform error checking, whereas code that simply assigns to a field may not go into the trouble. For example, a `setSalary` method might check that the salary is never less than 0.

---



**Caution:** Be careful not to write accessor methods that return references to mutable objects. In a previous edition of this book, I violated that rule in the `Employee` class in which the `getHireDay` method returned an object of class `Date`:

```
class Employee {
    private Date hireDay;
    . . .
    Date getHireDay() {
        return hireDay; // BAD
    }
    . . .
}
```

Unlike the `LocalDate` class, which has no mutator methods, the `Date` class has a mutator method, `setTime`, where you can set the number of

milliseconds.

The fact that `Date` objects are mutable breaks encapsulation! Consider the following rogue code:

```
Employee harry = . . .;
Date d = harry.getHireDay();
double tenYearsInMilliseconds = 10 * 365.25 * 24 * 60 * 60 * 1000;
d.setTime(d.getTime() - (long) tenYearsInMilliseconds);
// let's give Harry ten years of added seniority
```

The reason is subtle. Both `d` and `harry.hireDay` refer to the same object (see **Figure 4.5**). Applying mutator methods to `d` automatically changes the private state of the `Employee` object!

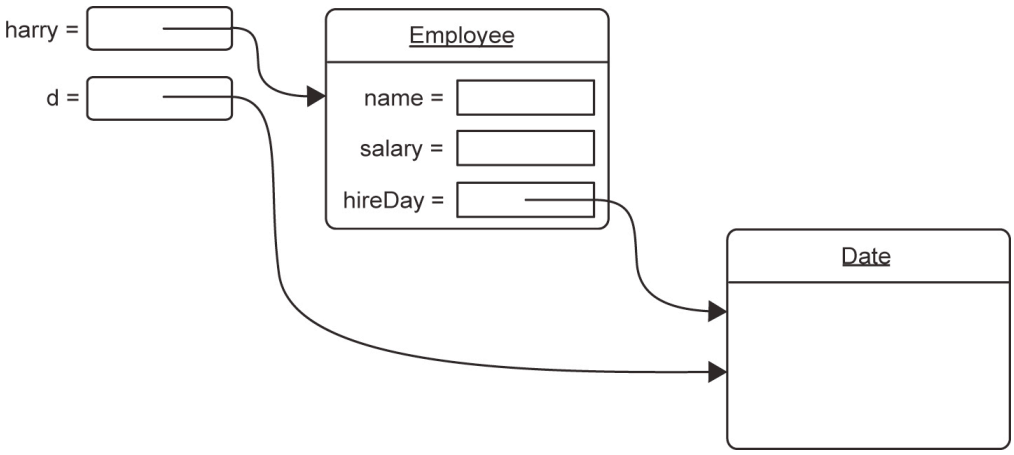



**Figure 4.5:** Returning a reference to a mutable instance field

You will see in **Chapter 6** how to solve this problem by *cloning* the mutable object before returning it.

A better remedy is to use immutable objects when possible, such as `LocalDate` instead of the legacy `Date` class.

### 4.3.8. Class-Based Access Privileges

You know that a method can access the private data of the object on which it is invoked. What people often find surprising is that a method can access the private data of *all objects of its class*. For example, consider a method `equals` that compares two employees.

```
class Employee {
    . . .
    public boolean equals(Employee other) {
```

```
        return id == other.id;
    }
}
```

A typical call is

```
if (harry.equals(boss)) . . .
```

This method accesses the private fields of `harry`, which is not surprising. It also accesses the private fields of `boss`. This is legal because `boss` is an object of type `Employee`, and a method of the `Employee` class is permitted to access the private fields of *any* object of type `Employee`.

### 4.3.9. Private Methods

When implementing a class, we make all instance fields private because public data are dangerous. But what about the methods? Sometimes, you may wish to break up the code for a computation into separate helper methods. Typically, these helper methods should not be part of the public interface—they may be too close to the current implementation or require a special protocol or calling order. Such methods are best implemented as private.

To implement a private method in Java, simply add the `private` modifier.

By making a method private, you are under no obligation to keep it available if you change your implementation. The method may well be *harder* to implement or *unnecessary* if the data representation changes; this is irrelevant. The point is that as long as the method is private, the designers of the class can be assured that it is never used elsewhere, so they can simply drop it. If a method is not private, you cannot simply drop it because other code might rely on it.

### 4.3.10. Final Instance Fields

You can define an instance field as `final`. Such a field must be initialized when the object is constructed. That is, you must guarantee that the field value has been set after the end of every constructor. Afterwards, the field may not be modified again. For example, the `name` field of the `Employee` class may be declared as `final` because it never changes after the object is constructed—there is no `setName` method.

```
class Employee {
    private final String name;
    . . .
```

```
}
```

The `final` modifier is particularly useful for fields whose type is primitive or an *immutable class*. (A class is immutable if none of its methods ever mutate its objects. For example, the `String` class is immutable.)

For mutable classes, the `final` modifier can be confusing. For example, consider a field

```
private final StringBuilder evaluations;
```

that is initialized in the `Employee` constructor as

```
evaluations = new StringBuilder();
```

The `final` keyword merely means that the object reference stored in the `evaluations` variable will never again refer to a different `StringBuilder` object. But the object can be mutated:

```
void giveGoldStar() {
    evaluations.append(LocalDate.now() + ": Gold star!\n");
}
```



**Note:** A `final` field can be `null`:

```
name = n != null && n.length() == 0 ? null : n;
```

Of course, it can then never be changed to a non-`null` value.

## 4.4. Static Fields and Methods

When you declare a class, you specify the data that each object stores, and the methods that work with the data. Sometimes, a class wants to specify data outside of objects, or methods that are not invoked on objects. These features are the topics of the following subsections.

### 4.4.1. Static Fields

If you define a field as `static`, then the field is not present in the objects of the class. There is only a single copy of each static field. You can think of static fields

as belonging to the class, not to the individual objects. For example, let's suppose we want to assign a unique identification number to each employee. We add an instance field `id` and a static field `nextId` to the `Employee` class:

```
class Employee {
    private static int nextId = 1;

    private int id;
    . . .
}
```

Every `Employee` object now has its own `id` field, but there is only one `nextId` field that is shared among all instances of the class. Let's put it another way. If there are 1,000 objects of the `Employee` class, then there are 1,000 instance fields `id`, one for each object. But there is a single static field `nextId`. Even if there are no `Employee` objects, the static field `nextId` is present. It belongs to the class, not to any individual object.

In the constructor, we assign the next available ID to the new `Employee` object and then increment it:

```
id = nextId;
nextId++;
```

Suppose we construct the object `harry`. Then the `id` field of `harry` is set to the current value of the static field `nextId`, and the value of the static field is incremented:

```
harry.id = Employee.nextId;
Employee.nextId++;
```

---



**Caution:** If you accidentally declare what you want to be an instance field as `static`, the compiler won't help you find your mistake.

Conceptually, a static field (such as `nextId` in the preceding example) belongs to the class, not to any object. Nevertheless, in Java, you can refer to the field as `Employee.nextId`, or `e.nextId`, where `e` is any `Employee` object. And inside the methods of the `Employee` class, you can use `nextId`, `this.nextId`, or `Employee.nextId`.

Therefore, it is a good idea to pay attention to any `static` declarations in a class. Static variables are uncommon and deserve close scrutiny. Static

constants, discussed in the following section, stand out because their names are typically in all caps.

### 4.4.2. Static Constants

Static variables are quite rare. However, static constants are more common. For example, the `Math` class defines a static constant:

```
public class Math {
    . . .
    public static final double PI = 3.14159265358979323846;
    . . .
}
```

You can access this constant in your programs as `Math.PI`.

If the keyword `static` had been omitted, then `PI` would have been an instance field of the `Math` class. That is, you would need an object of this class to access `PI`, and every `Math` object would have its own copy of `PI`.

The static constant `System.out` is declared in the `System` class as follows:

```
public class System {
    . . .
    public static final PrintStream out = . . .;
    . . .
}
```

As mentioned several times, fields should be `private`, because otherwise other code can modify them. However, public constants (that is, `final` fields) are fine. Since `out` has been declared as `final`, you cannot reassign another print stream to it:

```
System.out = new PrintStream(. . .); // ERROR--out is final
```



**Note:** If you look at the `System` class, you will notice a method `setOut` that sets `System.out` to a different stream. You may wonder how that method can change the value of a `final` variable. However, the `setOut` method is a *native* method, not implemented in the Java programming language. Native methods can bypass the access control mechanisms of the

Java language. This is a very unusual workaround that you should not emulate in your programs.

---



**Note:** For thirty years, invoking the `println` method on `System.out` was the preferred way to produce console output.

However, it is not easy to explain `System.out` to beginners. Why a public field? Why not uppercase `OUT` like all other public final fields in the Java API? Why native methods for changing it?

For ease of learning, Java 25 introduced the `IO.println` method instead.

---

### 4.4.3. Static Methods

Static methods are methods that do not operate on objects. For example, the `pow` method of the `Math` class is a static method. The expression

```
Math.pow(x, a)
```

computes the power $x^a$. It does not use any `Math` object to carry out its task. In other words, it has no implicit parameter.

You can think of static methods as methods that don't have a `this` parameter. (In a nonstatic method, the `this` parameter refers to the implicit parameter of the method—see **Section 4.3.6**.)

A static method of the `Employee` class cannot access the `id` instance field because it does not operate on an object. However, a static method can access a static field. Here is an example of such a static method:

```
static int advanceId() {
    int r = nextId; // obtain next available id
    nextId++;
    return r;
}
```

To call this method, you supply the name of the class:

```
int n = Employee.advanceId();
```

Could you have omitted the keyword `static` for this method? Yes, but then you would need to have an object reference of type `Employee` to invoke the method.



**Note:** It is legal to use an object to call a static method. For example, if `harry` is an `Employee` object, then you can call `harry.advanceId()` instead of `Employee.advanceId()`. However, I find that notation confusing. The `advanceId` method doesn't look at `harry` at all to compute the result. I recommend that you use class names, not objects, to invoke static methods. (Your IDE will probably suggest a refactoring otherwise.)

---

Use static methods in two situations:

- When a method doesn't need to access the object state because all needed parameters are supplied as explicit parameters (example: `Math.pow`)
- When a method only needs to access static fields of the class (example: `Employee.advanceId`)

---



**Note:** In some object-oriented programming languages, static fields and methods are called *class fields* and *class methods*.

The term "static" has a curious history. At first, the keyword `static` was introduced in C to denote local variables that don't go away when a block is exited. In that context, the term "static" makes sense: The variable stays around and is still there when the block is entered again. Then `static` got a second meaning in C, to denote global variables and functions that cannot be accessed from other files. The keyword `static` was simply reused to avoid introducing a new keyword. Finally, C++ reused the keyword for a third, unrelated, interpretation—to denote variables and functions that belong to a class but not to any particular object of the class. That is the same meaning the keyword has in Java.

---

### 4.4.4. Factory Methods

Here is another common use for static methods. Classes such as `LocalDate` and `NumberFormat` use static *factory methods* that construct objects. You have already seen the factory methods `LocalDate.now` and `LocalDate.of`. Here is how to obtain formatter objects for various styles:

```
NumberFormat currencyFormatter = NumberFormat.getCurrencyInstance();
NumberFormat percentFormatter = NumberFormat.getPercentInstance();
double x = 0.1;
IO.println(currencyFormatter.format(x)); // prints $0.10
IO.println(percentFormatter.format(x)); // prints 10%
```

Why doesn't the `NumberFormat` class use a constructor instead? There are three reasons to prefer a factory method over a constructor:

- You can't give names to constructors. The constructor name is always the same as the class name. But we want two different names to get the currency instance and the percent instance.
- When you use a constructor, you can't vary the type of the constructed object. But the factory methods actually return objects of the class `DecimalFormat`, a more specialized class that inherits from `NumberFormat`. (See **Chapter 5** for more on inheritance.)
- A constructor always constructs a new object. You may want to share instances. For example, the call `Set.of()` yields the same instance of an empty set when you call it twice.

### 4.4.5. The `main` Method

Note that you can call static methods without having any objects. For example, you never construct any objects of the `Math` class to call `Math.pow`.

For the same reason, the `main` method was traditionally a static method.

```
class Application {
    public static void main(String[] args) {
        // construct objects here
        . . .
    }
}
```

The static `main` method does not operate on any objects. After all, when a program starts, there aren't any objects yet. The `main` method executes and constructs the objects that the program needs.

As of Java 25, the `main` method no longer needs to be `public` or `static`, and it need not have a parameter of type `String[]` args. Here are the complete rules for `main` methods:

- If there is more than one `main` method, `static main` methods are preferred over instance methods
- Methods with a `String[]` parameter are preferred over those with no parameters.
- Private `main` methods are not considered.
- If `main` is not static, the class must have a non-private no-argument constructor (see **Section 4.6.3**). Then the launcher constructs an instance of the class and invokes the `main` method on it.

Also as of Java 25, a `main` method no longer needs to be declared inside a class. A source file with method declarations outside a class is a *compact compilation unit*.

It implicitly declares a class whose name is derived from the source file. For example, consider a source file with this content:

```
void main() { // top level method
    . . .
}
int counter = 0; // instance field
void work() { // another method
    counter++;
}
```

Technically, the choice of class name can depend on the host system. It will usually be the file name with the extension removed. If the file name is `Application.java`, the result is most likely a class with name `Application`. Top-level variables and methods become instance fields and methods of that class.

---



**Note:** You cannot declare a constructor for an implicitly declared class. After all, you can't completely rely on the name of the class, which is also the name of any constructor.

For the same reason, it doesn't make sense to implicitly declare classes that you want to use in other classes. You need a reliable name to use the class.

The practical use for a compact compilation unit is a class with a `main` method, which you launch as a source file.

---

Now let us turn to the next example program. The `Employee` class in **Listing 4.4** has a static field `nextId` and a static method `advanceId`. In the sample program (**Listing 4.3**), we fill an array with three `Employee` objects and then print the employee information. Finally, we print the next available identification number, to demonstrate the static method.

---



**Note:** In the code for this section, the `Employee` class was declared in a separate file `Employee.java`. In contrast, the compact compilation unit in the `EmployeeDemo.java` file of the preceding section contained both the `main` method of the program and the `Employee` class.

Technically, in that situation, the `Employee` class is an *inner* class of the implicit `EmployeeDemo` class. By using inner classes, we can present different variations of the `Employee` class in our sample programs. You will learn more about inner classes in **Chapter 6**.

---



**Tip:** Every class can have a `main` method. That can be handy for adding demonstration code to a class. For example, you can add a `main` method to the `Employee` class:

```
class Employee {
    Employee(String n, double s) {
        name = n;
        salary = s;
    }
    . . .
    static void main() { // runs demo
        var e = new Employee("Harry", 50000);
        IO.println(e.getName() + " " + e.getSalary());
    }
}
```

Note that in this example, `main` must be `static` since the `Employee` class doesn't have a no-argument constructor.

To see the demo of the `Employee` class, simply execute

```
java Employee.java
```

If the `Employee` class is a part of another program, its `main` method is not executed (unless, of course, it is explicitly called.)

**Listing 4.3 StaticDemo.java**

```
1  /**
2   * This program demonstrates static methods.
3   */
4  void main(String[] args) {
5      // fill the staff array with three Employee objects
6      var staff = new Employee[3];
7
8      staff[0] = new Employee("Tom", 40000);
9      staff[1] = new Employee("Dick", 60000);
10     staff[2] = new Employee("Harry", 65000);
11
12     // print out information about all Employee objects
13     for (Employee e : staff) {
14         IO.println("name=" + e.getName() + ",id=" + e.getId()
15                 + ",salary=" + e.getSalary());
16     }
17
```

```
    int n = Employee.advanceId(); // calls static method
    IO.println("Next issued id=" + n);
}
```

**Listing 4.4 Employee.java**

```
/**
 * A simplified employee class to demonstrate static fields and methods.
 * This class is used in StaticDemo.java and ParamDemo.java
 */
class Employee {
    private static int nextId = 1;

    private String name;
    private double salary;
    private int id;

    Employee(String n, double s) {
        name = n;
        salary = s;
        id = advanceId();
    }

    String getName() {
        return name;
    }

    double getSalary() {
        return salary;
    }

    int getId() {
        return id;
    }

    void raiseSalary(double byPercent) {
        double raise = salary * byPercent / 100;
        salary += raise;
    }

    static int advanceId() {
        int r = nextId; // obtain next available id
        nextId++;
        return r;
    }

    static void main() { // runs demo
        var e = new Employee("Harry", 50000);
```

```
        IO.println(e.getName() + " " + e.getSalary());
    }
}
```

`java.util.Objects` 7

- `static <T> void requireNonNull(T obj)`
- `static <T> void requireNonNull(T obj, String message)`
- `static <T> void requireNonNull(T obj, Supplier<String> messageSupplier)` 8

  If `obj` is `null`, these methods throw a `NullPointerException` with no message or the given message. (**Chapter 6** explains how to obtain a value lazily with a supplier. **Chapter 8** explains the `<T>` syntax.)
- `static <T> T requireNonNullElse(T obj, T defaultObj)` 9
- `static <T> T requireNonNullElseGet(T obj, Supplier<T> defaultSupplier)` 9

  return `obj` if it is not `null`, or the default object if `obj` is `null`.

## 4.5. Method Parameters

Let us review the computer science terms that describe how parameters can be passed to a method (or a function) in a programming language. The term *call by value* means that the method gets just the value that the caller provides. In contrast, *call by reference* means that the method gets the *location* of the variable that the caller provides. Thus, a method can *modify* the value stored in a variable passed by reference but not in one passed by value. These "call by . . ." terms are standard computer science terminology describing the behavior of method parameters in various programming languages, not just Java. (There is also a *call by name* that is mainly of historical interest, being employed in the Algol programming language, one of the oldest high-level languages.)

The Java programming language *always* uses call by value. That means that the method gets a copy of all arguments. In particular, the method cannot modify the contents of any variables in the method call.

For example, consider the following call:

```
double percent = 10;
harry.raiseSalary(percent);
```

No matter how the method is implemented, we know that after the method call, the value of `percent` is still `10`.

Let us look a little more closely at this situation. Suppose a method tried to triple the value of a method parameter:

```
void tripleValue(double x) { // doesn't work
    x = 3 * x;
}
```

Let's call this method:

```
double percent = 10;
tripleValue(percent);
```

However, this does not work. After the method call, the value of `percent` is still `10`. Here is what happens:

1. `x` is initialized with a copy of the value of `percent` (that is, `10`).
2. `x` is tripled—it is now `30`. But `percent` is still `10` (see **Figure 4.6**).

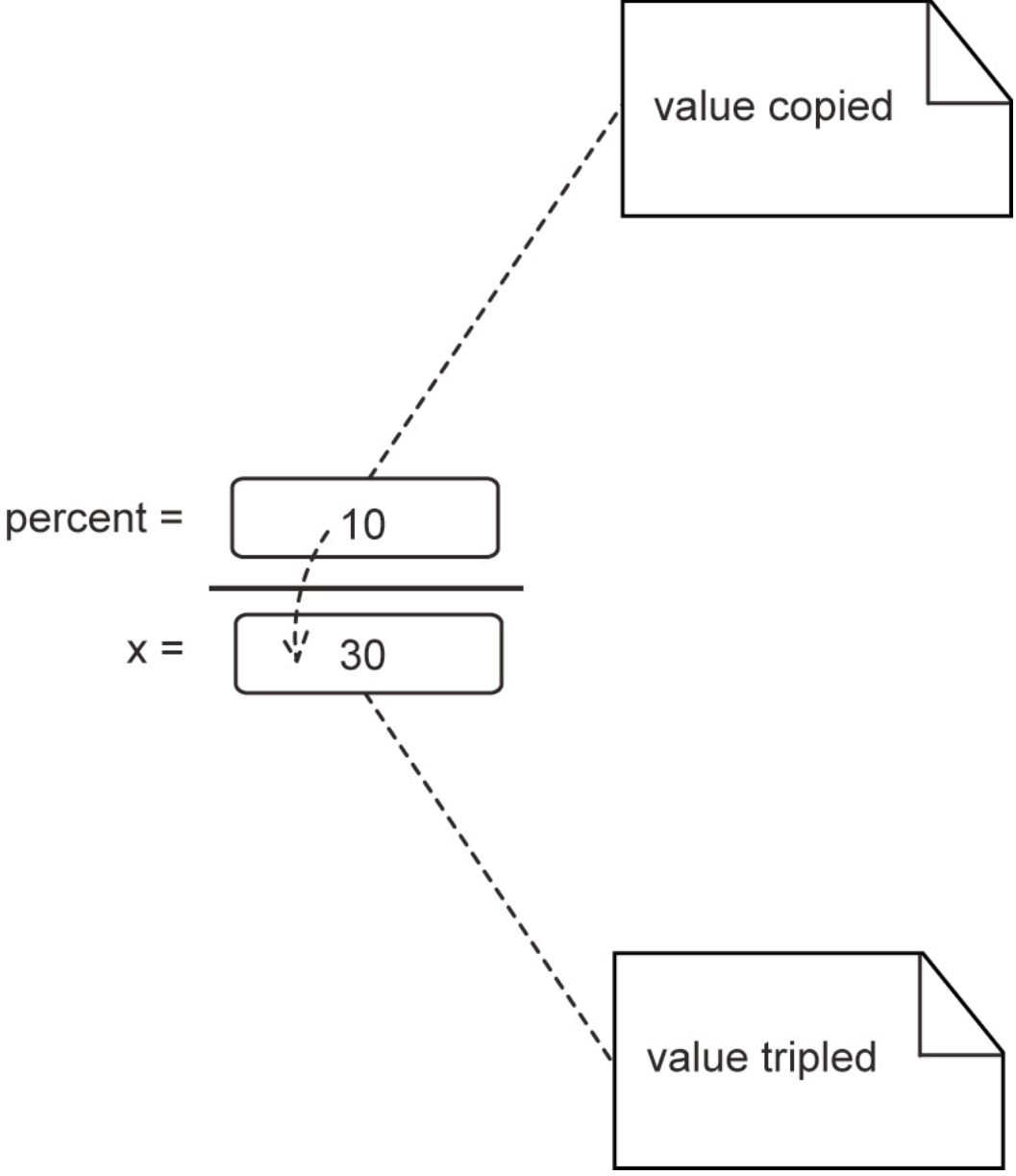



**Figure 4.6:** Modifying a parameter variable has no lasting effect.

3. The method ends, and the parameter variable `x` is no longer in use.

There are, however, two kinds of method parameters:

- Primitive types (number types, `char`, `boolean`)
- Object references

You have seen that it is impossible for a method to change a primitive type parameter. The situation is different for object parameters. You can easily implement a method that triples the salary of an employee:

```
void tripleSalary(Employee x) { // works
    x.raiseSalary(200);
}
```

When you call

```
harry = new Employee(. . .);
tripleSalary(harry);
```

then the following happens:

1. `x` is initialized with a copy of the value of `harry`—that is, an object reference.
2. The `raiseSalary` method is applied to that object reference. The `Employee` object to which both `x` and `harry` refer gets its salary raised by 200 percent.
3. The method ends, and the parameter variable `x` is no longer in use. Of course, the object variable `harry` continues to refer to the object whose salary was tripled (see **Figure 4.7**).

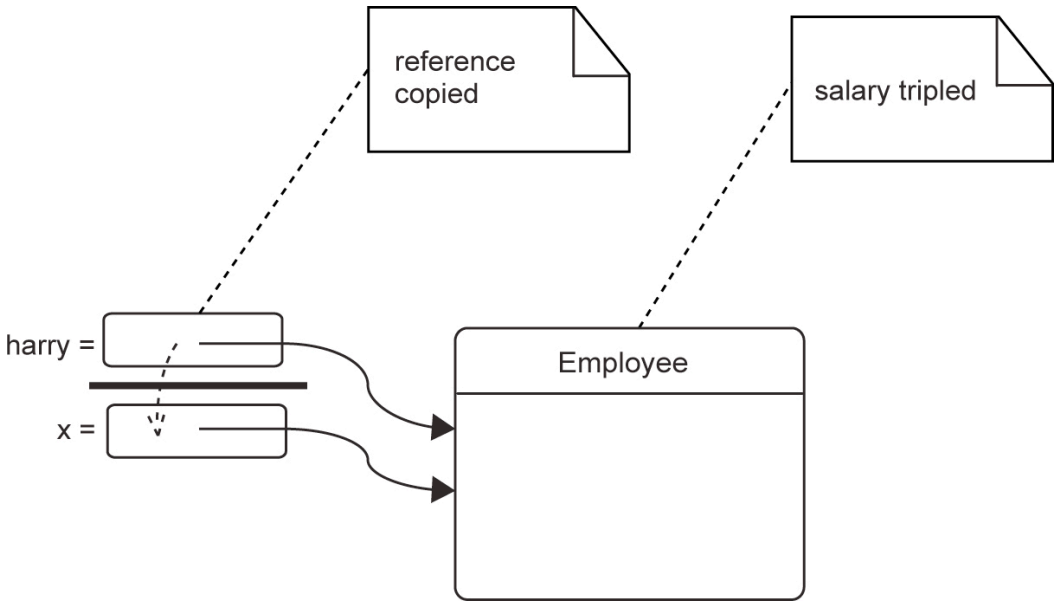



**Figure 4.7:** Modifying an object referenced by a parameter has a lasting effect.

As you have seen, it is easily possible—and in fact very common—to implement methods that change the state of an object parameter. The reason is simple. The method gets a copy of the object reference, and both the original and the copy refer to the same object.

Many programming languages (in particular, C++ and Pascal) have two mechanisms for parameter passing: call by value and call by reference. Some programmers (and unfortunately even some book authors) claim that Java uses call by reference for objects. That is false. As this is such a common misunderstanding, it is worth examining a counterexample in detail.

Let's try to write a method that swaps two `Employee` objects:

```
void swap(Employee x, Employee y) { // doesn't work
    Employee temp = x;
    x = y;
    y = temp;
}
```

If Java used call by reference for objects, this method would work:

```
var a = new Employee("Alice", . . .);
var b = new Employee("Bob", . . .);
swap(a, b);
// does a now refer to Bob, b to Alice?
```

However, the method does not actually change the object references that are stored in the variables `a` and `b`. The `x` and `y` parameters of the `swap` method are initialized with *copies* of these references. The method then proceeds to swap these copies.

```
// x refers to Alice, y to Bob
Employee temp = x;
x = y;
y = temp;
// now x refers to Bob, y to Alice
```

But ultimately, this is a wasted effort. When the method ends, the parameter variables `x` and `y` are abandoned. The original variables `a` and `b` still refer to the same objects as they did before the method call (see **Figure 4.8**).

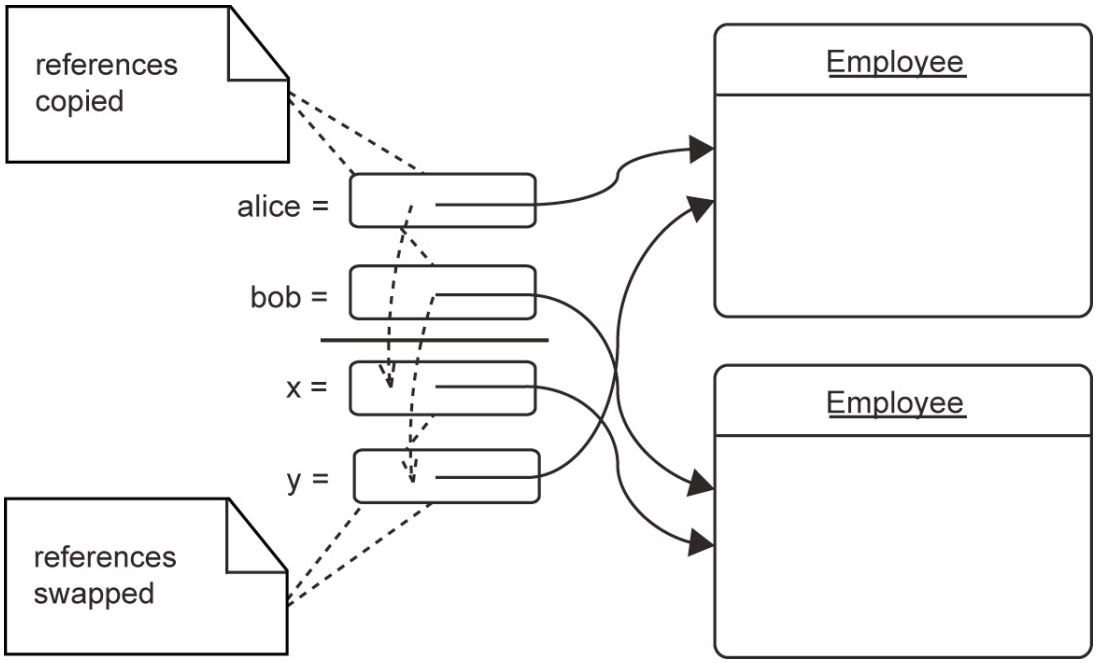



**Figure 4.8:** Swapping parameter variables has no lasting effect.

This demonstrates that the Java programming language does not use call by reference for objects. Instead, *object references are passed by value.*

Here is a summary of what you can and cannot do with method parameters in Java:

- A method cannot modify a parameter of a primitive type (that is, number types, `char`, or `boolean`).
- A method can change the *state* of an object parameter.
- A method cannot make an object parameter refer to a new object.

The program in **Listing 4.5** demonstrates these facts. The program first tries to triple the value of a number parameter and does not succeed:

```
Testing tripleValue:
Before: percent=10.0
End of method: x=30.0
After: percent=10.0
```

It then successfully triples the salary of an employee:

```
Testing tripleSalary:
Before: salary=50000.0
End of method: salary=150000.0
After: salary=150000.0
```

After the method, the state of the object to which `harry` refers has changed. This is possible because the method modified the state through a copy of the object reference.

Finally, the program demonstrates the failure of the `swap` method:

```
Testing swap:
Before: a=Alice
Before: b=Bob
End of method: x=Bob
End of method: y=Alice
After: a=Alice
After: b=Bob
```

As you can see, the parameter variables `x` and `y` are swapped, but the variables `a` and `b` are not affected.



**Note:** Some programming languages have both call by value and call by reference. In C++, you tag reference parameters with `&`. For example, you can easily implement methods `void tripleValue(double& x)` or `void swap(Employee& x, Employee& y)` that modify their reference parameters.

In C#, reference parameters are declared as `ref` (bidirectional), `in`, or `out`.

**Listing 4.5 ParamDemo.java**

```
/**
 * This program demonstrates parameter passing in Java.
 */
void main() {
    /*
     * Test 1: Methods can't modify numeric parameters
     */
    IO.println("Testing tripleValue:");
    double percent = 10;
    IO.println("Before: percent=" + percent);
    tripleValue(percent);
    IO.println("After: percent=" + percent);

    /*
     * Test 2: Methods can change the state of object parameters
     */
    IO.println("\nTesting tripleSalary:");
    var harry = new Employee("Harry", 50000);
    IO.println("Before: salary=" + harry.getSalary());
    tripleSalary(harry);
    IO.println("After: salary=" + harry.getSalary());

    /*
     * Test 3: Methods can't attach new objects to object parameters
     */
    IO.println("\nTesting swap:");
    var a = new Employee("Alice", 70000);
    var b = new Employee("Bob", 60000);
    IO.println("Before: a=" + a.getName());
    IO.println("Before: b=" + b.getName());
    swap(a, b);
    IO.println("After: a=" + a.getName());
    IO.println("After: b=" + b.getName());
}

void tripleValue(double x) { // doesn't work
    x = 3 * x;
```

```
    IO.println("End of method: x=" + x);
}

void tripleSalary(Employee x) { // works
    x.raiseSalary(200);
    IO.println("End of method: salary=" + x.getSalary());
}

void swap(Employee x, Employee y) {
    Employee temp = x;
    x = y;
    y = temp;
    IO.println("End of method: x=" + x.getName());
    IO.println("End of method: y=" + y.getName());
}
```

## 4.6. Object Construction

You have seen how to write simple constructors that define the initial state of your objects. However, since object construction is so important, Java offers quite a variety of mechanisms for writing constructors. We go over these mechanisms in the sections that follow.

### 4.6.1. Overloading

Some classes have more than one constructor. For example, you can construct an empty `StringBuilder` object as

```
var messages = new StringBuilder();
```

Alternatively, you can specify an initial string:

```
var todoList = new StringBuilder("To do:\n");
```

This capability is called *overloading*. Overloading occurs if several methods have the same name (in this case, the `StringBuilder` constructor method) but different parameters. The compiler must sort out which method to call. It picks the correct method by matching the parameter types in the declarations of the various methods with the types of the arguments used in the specific method call. A compile-time error occurs if the compiler cannot match the parameters, either because there is no match at all or because there is not one that is better than all others. (The process of finding a match is called *overloading resolution*.)



**Note:** Java allows you to overload any method—not just constructor methods. Thus, to completely describe a method, you need to specify its name together with its parameter types. This is called the *signature* of the method. For example, the `String` class has six public methods called `indexOf`. They have signatures

```
indexOf(int)
indexOf(int, int)
indexOf(int, int, int)
indexOf(String)
indexOf(String, int)
indexOf(String, int, int)
```

The return type is not part of the method signature. That is, you cannot have two methods with the same names and parameter types but different return types.



**Caution:** The statement

```
StringBuilder builder = new StringBuilder('\n'); // ERROR
```

does *not* produce a `StringBuilder` containing a newline. There is a constructor with an `int` parameter, which yields a `StringBuilder` with a preallocated capacity. The `char` literal `'\n'` can be converted to an `int` but not to a `String`. The result is a builder with no contents and capacity 10, the integer value of `'\n'`.

In general, be careful with overloading when the argument types do not match the parameter types exactly.

### 4.6.2. Default Field Initialization

If you don't set a field explicitly in a constructor, it is automatically set to a default value: numbers to `0`, `boolean` values to `false`, and object references to `null`. Some people consider it poor programming practice to rely on the defaults. Certainly, it makes it harder for someone to understand your code if fields are being initialized invisibly.



**Note:** This is an important difference between fields and local variables. You must always explicitly initialize local variables in a method. But in a class, if you don't initialize a field, it is automatically initialized to a default (`0`, `false`, or `null`).

For example, consider the `Employee` class. Suppose you don't specify how to initialize some of the fields in a constructor. By default, the `salary` field would be initialized with `0` and the `name` and `hireDay` fields would be initialized with `null`.

However, that would not be a good idea. If anyone called the `getName` or `getHireDay` method, they would get a `null` reference that they probably don't expect:

```
LocalDate h = harry.getHireDay();
int year = h.getYear(); // throws exception if h is null
```

### 4.6.3. The Constructor with No Arguments

Many classes contain a constructor with no arguments that creates an object whose state is set to an appropriate default. For example, here is a no-argument constructor for the `Employee` class:

```
Employee() {
    name = "";
    salary = 0;
    hireDay = LocalDate.now();
}
```

If you write a class with no constructors whatsoever, then a no-argument constructor is provided for you. This constructor sets *all* the instance fields to their default values. So, all numeric data contained in the instance fields would be `0`, all `boolean` values would be `false`, and all object variables would be `null`.

If a class supplies at least one constructor but does not supply a no-argument constructor, it is illegal to construct objects without supplying arguments. For example, our original `Employee` class in **Listing 4.2** provided a single constructor:

```
Employee(String n, double s, int year, int month, int day)
```

With that class, it was not legal to construct default employees. That is, the call

```
e = new Employee();
```

would have been an error.



**Caution:** Please keep in mind that you get a free no-argument constructor *only* when your class has no other constructors. If you write your class with even a single constructor of your own and you want the users of your class to have the ability to create an instance by a call to

```
new ClassName()
```

then you must provide a no-argument constructor. Of course, if you are happy with the default values for all fields, you can simply supply a constructor with an empty body:

```
ClassName() { }
```



**Note:** Some object-oriented programming languages, notably C++, have explicit destructor methods for any cleanup code that may be needed when an object is no longer used. The most common activity in a destructor is reclaiming the memory set aside for objects. Since Java does automatic garbage collection, manual memory reclamation is not needed, so Java does not support destructors.

Of course, some objects utilize a resource other than memory, such as a file or a handle to another object that uses system resources. In this case, it is important that the resource be reclaimed and recycled when it is no longer needed.

The `Object` class has a `finalize` method that classes can override for cleanup. That method was intended to be called before the garbage collector sweeps away an object. However, you simply cannot know when this method will be called, and it is now deprecated for removal.

If a resource needs to be closed as soon as you have finished using it, supply a `close` method that does the necessary cleanup. You can call the `close` method when you are done with the object. In **Chapter 7**, you will see how you can ensure that this method is called automatically.

### 4.6.4. Explicit Field Initialization

By overloading the constructor methods in a class, you can build many ways to set the initial state of the instance fields of your classes. It is always a good idea to make sure that, regardless of the constructor call, every instance field is set to something meaningful.

You can simply assign a value to any field in the class definition. For example:

```
class Employee {
    private String name = "";
    . . .
}
```

This assignment is carried out before the constructor executes. This syntax is particularly useful if all constructors of a class need to set a particular instance field to the same value.

The initialization value doesn't have to be a constant value. Here is an example in which a field is initialized with a method call. Consider the `Employee` class where each employee has an `id` field. You can initialize it as follows:

```
class Employee {
    private static int nextId;
    private int id = advanceId();
    . . .
    private static int advanceId() {
        int r = nextId;
        nextId++;
        return r;
    }
    . . .
}
```

### 4.6.5. Parameter Names

When you write very trivial constructors (and you'll write a lot of them), it can be somewhat frustrating to come up with parameter names.

We have generally opted for single-letter parameter names:

```
Employee(String n, double s) {
    name = n;
    salary = s;
}
```

However, the drawback is that you need to read the code to tell what the `n` and `s` parameters mean.

Some programmers prefix each parameter with an "`a`":

```
Employee(String aName, double aSalary) {
    name = aName;
    salary = aSalary;
}
```

That is better. Any reader can immediately figure out the meaning of the parameters.

Another commonly used trick relies on the fact that parameter variables *shadow* instance fields with the same name. For example, if you call a parameter `salary`, then `salary` refers to the parameter, not the instance field. But you can still access the instance field as `this.salary`. Recall that `this` denotes the implicit parameter—that is, the object being constructed. Here is an example:

```
Employee(String name, double salary) {
    this.name = name;
    this.salary = salary;
}
```



**Note:** Some programmers like to prefix instance fields with an underscore or a fixed letter. (The letters `m` and `x` are common choices.) For example, the salary field might be called `_salary`, `mSalary`, or `xSalary`. However, this style is not common in Java.

### 4.6.6. Calling Another Constructor

The keyword `this` refers to the implicit parameter of a method. However, this keyword has a second meaning.

A constructor can call another constructor of the same class. Here is a typical example:

```
Employee(double s) {
    // calls Employee(String, double)
    this("Employee #" + nextId, s);
    nextId++;
}
```

When you call `new Employee(60000)`, the `Employee(double)` constructor calls the `Employee(String, double)` constructor.

By having one constructor call another, you can often eliminate duplicate construction code.

Before Java 25, the call to the other constructor had to be the *first statement* of the constructor body. This restriction has now been removed.

However there are some restrictions on what can happen between the start of a constructor and the call of another constructor. This phase is called the *early construction context*.

In the early construction context, you may not:

- Read any instance variable
- Write any instance variable that has an explicit initialization
- Invoke any methods on `this`
- Pass `this` to any other methods

Due to these restrictions, no code can observe a partially constructed object in the early construction context.

One useful action in the early construction context is to validate or normalize construction parameters:

```
Employee(double s) {
    if (s < 0) s = 0; // Cannot have negative salary
    this("Employee #" + nextId, s);
    nextId++;
}
```

### 4.6.7. Initialization Blocks

You have already seen two ways to initialize an instance field:

- By setting a value in a constructor
- By assigning a value in the declaration

There is a third mechanism in Java, called an *initialization block*. Class declarations can contain arbitrary blocks of code. These blocks are executed whenever an object of that class is constructed. For example:

```
class Employee {
    private static int nextId;

    private int id;
    private String name;
    private double salary;

    // object initialization block
```

```
    {
        id = nextId;
        nextId++;
    }

    Employee(String n, double s) {
        name = n;
        salary = s;
    }

    Employee() {
        name = "";
        salary = 0;
    }
    . . .
}
```

In this example, the `id` field is initialized in the object initialization block, no matter which constructor is used to construct an object. The initialization block runs first, and then the body of the constructor is executed.

This mechanism is never necessary and is not common. It is usually more straightforward to place the initialization code inside a constructor.

---



**Note:** It is legal to set fields in initialization blocks even if they are only defined later in the class. However, to avoid circular definitions, it is not legal to read from fields that are only initialized later. The exact rules are spelled out in Section 8.3.3 of the Java Language Specification (**https://docs.oracle.com/javase/specs**). The rules are complex enough to baffle the compiler implementors—early versions of Java implemented them with subtle errors. Therefore, you should always place initialization blocks after the field definitions.

---

To summarize, here is what happens in detail when a constructor is called:

1. All instance fields are initialized to their default values (`0`, `false`, or `null`).
2. All field initializers and initialization blocks are executed, in the order in which they occur in the class declaration.
3. The body of the constructor is executed.
4. If the constructor invokes another constructor, the body of that constructor executes as well.

### 4.6.8. Static Initialization

To initialize a static field, either supply an initial value or use a static initialization block. You have already seen the first mechanism:

```
private static int nextId = 1;
```

If the static fields of your class require complex initialization code, use a static initialization block.

Place the code inside a block and tag it with the keyword `static`. Here is an example. We want the employee ID numbers to start at a random integer less than 10,000.

```
private static RandomGenerator generator = RandomGenerator.getDefault();
// static initialization block
static {
    nextId = generator.nextInt(10000);
}
```

Static initialization occurs when the class is first loaded. Like instance fields, static fields are `0`, `false`, or `null` unless you explicitly set them to another value. All static field initializers and static initialization blocks are executed in the order in which they occur in the class declaration.

---



**Note:** Amazingly enough, up to Java 6, it was possible to write a "Hello, World" program in Java without ever writing a `main` method.

```
public class Hello {
    static {
        System.out.println("Hello, World");
    }
}
```

When you invoked the class with `java Hello`, the class was loaded, the static initialization block printed "Hello, World", and only then was a message displayed that `main` is not defined. Since Java 7, the `java` program first checks that there is a `main` method.

---



**Caution:** It is possible to have cycles in static initialization. Consider these classes:

```
class Config {
    static final Config DEFAULT = new Config();
    String get(String key) { . . . }
}

class Logger {
    static final Logger DEFAULT
        = new Logger(Config.DEFAULT.get("logger.default.file"));
    void log(String message) { . . . }
}
```

Now suppose the `Config` constructor adds a logging message:

```
Config() {
    // read configuration
    Logger.DEFAULT.log("Config read successfully");
}
```

The first time you use a `Logger`, the static initializization of the `Config` class invokes the `Config` constructor. It calls the `log` method on the `Logger.DEFAULT` variable, which has not yet been set. A `NullPointerException` occurs, which causes a fatal `ExceptionInInitializerError`.

Such errors are not common, but when they occur, they can be challenging to debug.

---

The program in **Listing 4.6** shows many of the features discussed in this section:

- Overloaded constructors
- A call to another constructor with `this(. . .)`
- A no-argument constructor
- An object initialization block
- A static initialization block
- An instance field initialization

The example uses a `RandomGenerator` instance for generating random numbers. Since JDK 17, the `java.util.random` package provides implementations of strong algorithms with various tradeoffs. There is a default that works well for most purposes, but if you are interested in alternatives, read through the API documentation of the `java.util.random` package for advice on which algorithm to choose.

Generate random numbers by calling `generator.nextInt(n)` or other `RandomGenerator` methods. (Technically, `RandomGenerator` is an *interface*, a concept introduced in **Chapter 6**. You don't need to worry about that when you invoke methods on a `RandomGenerator` instance.)



**Tip:** The `RandomGenerator` interface improves upon the `Random` class from Java 1.0, which has a fairly short *period* of $2^{48}$, after which the internal state repeats. As a rule of thumb, to generate $n$ random numbers, the period should be at least $n^2$. By that rule, the classic `Random` generator is unsuitable if $n$ exceeds $2^{24}$, which is not that large—about 17 million. These days, much better algorithms are known, and it is a good idea to move away from the `Random` class.

**Listing 4.6 ConstructorDemo.java**

```
/**
 * This program demonstrates object construction.
 */
void main() {
    // fill the staff array with three Employee objects
    var staff = new Employee[3];

    staff[0] = new Employee("Harry", 40000);
    staff[1] = new Employee(60000);
    staff[2] = new Employee();

    // print out information about all Employee objects
    for (Employee e : staff) {
        IO.println("name=" + e.getName() + ",id=" + e.getId()
                + ",salary=" + e.getSalary());
    }
}

/**
 * This version of the Employee class demonstrates overloaded constructors
 * and initialization blocks.
 */
class Employee {
    private static int nextId;

    private int id;
    private String name = ""; // instance field initialization
    private double salary;

    private static RandomGenerator generator = RandomGenerator.getDefault();

    // static initialization block
```

```
    static {
        // set nextId to a random number between 0 and 9999
        nextId = generator.nextInt(10000);
    }

    // object initialization block
    {
        id = nextId;
        nextId++;
    }

    // three overloaded constructors
    public Employee(String n, double s) {
        name = n;
        salary = s;
    }

    public Employee(double s) {
        // calls the Employee(String, double) constructor
        this("Employee #" + nextId, s);
    }

    // the no-argument constructor
    public Employee() {
        // name initialized to ""--see above
        // salary not explicitly set--initialized to 0
        // id initialized in initialization block
    }

    public String getName() {
        return name;
    }

    public double getSalary() {
        return salary;
    }

    public int getId() {
        return id;
    }
}
```

`java.util.random.RandomGenerator` 17

- `int nextInt(int n)`
  returns a random integer between 0 and `n` – 1.

- `static RandomGenerator of(String name)`
  yields a random generator for the given algorithm name. The algorithm named `"L64X128MixRandom"` is suitable for most applications.
- `static RandomGenerator getDefault()`
  yields a strong random generator with 64 or more state bits.

`java.util.Random` 1.0

- `static from(RandomGenerator generator)` 19
  returns a `Random` instance whose methods delegate to the given `RandomGenerator`. This allows legacy code to use strong random number generators.

## 4.7. Records

Sometimes, data is just data, and the data hiding that object-oriented programming provides gets in the way. Consider a class `Point` that describes a point in the plane, with x- and y-coordinates.

Sure, you can create a class:

```
class Point {
    private final double x;
    private final double y;

    Point(double x, double y) { this.x = x; this.y = y; }
    double getX() { return x; }
    double getY() { return y; }
    // More methods . . .
}
```

But does it really buy us anything to hide `x` and `y`, and then make the values available through the getter methods?

Would we ever want to change the implementation of a `Point`? Sure, there are polar coordinates, but you would not use them with a graphics API. In practice, a point in the plane is completely described by its x- and y-coordinates.

To define such classes more concisely, JDK 14 introduced “records” as a preview feature. The final version was delivered in JDK 16.

### 4.7.1. The Record Concept

A record is a special form of a class whose state is immutable and readable by the public. To declare a record, provide the name and the instance fields that hold the object state. Here is how you define `Point` as a record:

```
record Point(double x, double y) { }
```

The result is a class with instance fields:

```
private final double x;
private final double y;
```

In the Java language specification, the instance fields of a record are called its *components*.

The class has a constructor

```
Point(double x, double y)
```

and public accessor methods

```
double x()
double y()
```

Note that the accessors are called `x` and `y`, not `getX` and `getY`. (It is legal in Java to have an instance field and a method with the same name.)

```
var p = new Point(3, 4);
IO.println(p.x() + " " + p.y());
```

---



**Note:** Records don't follow the `get` convention because it is a bit messy. For `boolean` fields, it is common to use `is` instead of `get`. And the capitalization of the first letter can be problematic. What should happen if a class has fields `x` and `X`? Some programmers are unhappy because their legacy classes cannot trivially become records. But in practice, many of those legacy classes are mutable and therefore not candidates for conversion to records.

---

In addition to the field accessor methods, every record has three methods defined automatically: `toString`, `equals`, and `hashCode`. You will learn more about these methods in **Chapter 5**.

You can add your own methods to a record:

```
record Point(double x, double y) {
    double distanceFromOrigin() { return Math.hypot(x, y); }
}
```

---



**Caution:** You can define your own versions of the automatically provided methods, as long as they have the same parameter and return types. For example, this definition is legal:

```
record Point(double x, double y) {
    public double x() { return 2 * x; } // BAD
}
```

But it is surely not a good idea.

---

You cannot add additional instance fields to a record.

```
record Point(double x, double y) {
    private double z; // ERROR
    . . .
}
```

---



**Caution:** Instance fields of a record are automatically `final`. However, they may be references to mutable objects:

```
record PointInTime(double x, double y, Date when) { }
```

Then record instances are mutable:

```
var pt = new PointInTime(0, 0, new Date());
pt.when().setTime(0);
```

If you intend record instances to be immutable, don't use mutable types for fields.

---



**Note:** As you will see in detail in **Chapter 6**, you can declare a record inside another class. Then the enclosing class has access to the fields. For example, enclosing class code can refer to `p.x` instead of `p.x()`, where `p` is a variable of a nested record `Point`.

Note that this applies to any record in a compact compilation unit, since it is nested inside the implicitly declared class.

---

A record, like any class, can have static fields and methods:

```
record Point(double x, double y) {
    static Point ORIGIN = new Point(0, 0);
    static double distance(Point p, Point q) {
        return Math.hypot(p.x - q.x, p.y - q.y);
    }
    . . .
}
```

---



**Tip:** Use a record instead of a class for immutable data that is completely represented by a set of variables. Use a class if the data is mutable, or if the representation may evolve over time. Records are easier to read, more efficient, and safer in concurrent programs.

---

### 4.7.2. Constructors: Canonical, Compact, and Custom

The automatically defined constructor that sets all instance fields is called the *canonical constructor*.

To validate or normalize the parameters for the canonical constructor, use a *compact* form. Don't specify the parameter list:

```
record Range(int from, int to) {
    Range { // Compact form, validating parameters
        if (from > to) throw new IllegalArgumentException();
    }
}
```

or

```
record Range(int from, int to) {
    Range { // Compact form, normalizing parameters
        if (from > to) { // Swap the bounds
            int temp = from;
            from = to;
```

```
            to = temp;
        }
    }
}
```

The body of the compact form is the "prelude" to the canonical constructor. It merely modifies the parameter variables `from` and `to` before they are assigned to the instance fields `this.from` and `this.to`. You cannot read or modify the instance fields in the body of the compact constructor.

You can define additional *custom constructors*. A custom constructor must call another constructor, so that ultimately the canonical constructor is invoked. Here is an example:

```
record Point(double x, double y) {
    Point() { this(0, 0); }
}
```

This record has two constructors: the canonical constructor and a no-argument constructor yielding the origin.

---



**Note:** Before Java 25, the call to the other constructor had to be the first statement. Nowadays, you can have code in the early execution context, before the call to the other constructor. **Section 4.6.6** describes what you can do in that context.

---

If the canonical constructor needs to do additional work, you can provide your own implementation. But this is very uncommon. For example, the following implementation is valid, but, as you have seen, the same effect is easier to achieve with a compact constructor.

```
record Range(int from, int to) {
    Range(int from, int to) { // legal, but more work than a compact constructor
        if (from <= to) {
            this.from = from;
            this.to = to;
        }
        else {
            this.from = to;
            this.to = from;
        }
    }
}
```

**Listing 4.7 RecordDemo.java**

```
/**
 * This program demonstrates records.
 */
void main(String[] args) {
    var p = new Point(3, 4);
    IO.println("Coordinates of p: " + p.x() + " " + p.y());
    IO.println("Distance from origin: " + p.distanceFromOrigin());
    // Same computation with static field and method
    IO.println("Distance from origin: " + Point.distance(Point.ORIGIN, p));

    // Invoking a compact constructor
    var r = new Range(4, 3);
    IO.println("r: " + r);

    // A mutable record
    var pt = new PointInTime(3, 4, new Date());
    IO.println("Before: " + pt);
    pt.when().setTime(0);
    IO.println("After: " + pt);
}

record Point(double x, double y) {
    // A custom constructor
    Point() {
        this(0, 0);
    }

    // A method
    double distanceFromOrigin() {
        return Math.hypot(x, y);
    }

    // A static field and method
    static Point ORIGIN = new Point();

    static double distance(Point p, Point q) {
        return Math.hypot(p.x - q.x, p.y - q.y);
    }
}

record Range(int from, int to) {
    // A compact constructor
    Range {
        if (from > to) { // Swap the bounds
            int temp = from;
            from = to;
            to = temp;
        }
```

```
49      }
50  }
51
52  record PointInTime(double x, double y, Date when) { }
```

## 4.8. Packages

Java allows you to group classes in a collection called a *package*. Packages are convenient for organizing your work and for separating your work from code libraries provided by others. In the following sections, you will learn how to use and create packages.

### 4.8.1. Encapsulation

You already know how classes can hide implementation details, by declaring fields and methods as `private`. Packages provide a similar mechanism, but encapsulation is the default. Only classes that are declared as `public` can be used in other packages, and only methods that are declared as `public` can be called by other packages.

A class or method that is not declared `public` can only be accessed by code from the same package.

If we want to make our `Employee` class and its methods available to other packages, we should declare it as follows:

```
public class Employee {
    private String name;
    private double salary;
    private LocalDate hireDay;

    public Employee(String name, double salary, int year, int month, int day) {
        . . .
    }

    public String getName() {
        . . .
    }
    . . .
}
```

### 4.8.2. Package Names

One reason for using packages is to guarantee the uniqueness of class names. Suppose two programmers come up with the bright idea of supplying an

`Employee` class. As long as both of them place their class into different packages, there is no conflict. In fact, to absolutely guarantee a unique package name, use an Internet domain name (which is known to be unique) written in reverse. You then use subpackages for different projects. For example, consider the domain `horstmann.com`. When written in reverse order, it turns into the package name `com.horstmann`. You can then append a project name, such as `com.horstmann.corejava`. If you then place the `Employee` class into that package, the *qualified* name becomes `com.horstmann.corejava.Employee`.



**Note:** From the point of view of the compiler, there is absolutely no relationship between nested packages. For example, the packages `java.util` and `java.util.random` have nothing to do with each other. Each is its own independent collection of classes.

### 4.8.3. Class Importation

A class can use all classes from its own package and all classes declared as `public` from other packages.

You can access the public classes in another package in two ways. The first is simply to use the *qualified name*; that is, the package name followed by the class name. For example:

```
java.time.LocalDate today = java.time.LocalDate.now();
```

That is obviously tedious. A simpler, and more common, approach is to use the `import` statement. The point of the `import` statement is to give you a shorthand to refer to the classes in the package. Once you add an `import`, you no longer have to give the classes their full names.

You can import a specific class or the whole package. You place `import` statements at the top of your source files (but below any `package` statements). For example, you can import all classes in the `java.time` package with the statement

```
import java.time.*;
```

Then you can use

```
LocalDate today = LocalDate.now();
```

without a package prefix. You can also import a specific class inside a package:

```
import java.time.LocalDate;
```

The `java.time.*` syntax is less tedious. It has no negative effect on code size. However, if you import classes explicitly, the reader of your code knows exactly which classes you use.

However, note that you can only use the `*` notation to import a single package. You cannot use `import java.*` or `import java.*.*` to import all packages with the `java` prefix.

---



**Note:** You never need to import classes from the `java.lang` package.

---



**Tip:** Integrated development environments have commands to organize imports. Package statements such as `import java.util.*;` are automatically expanded into a list of specific imports such as

```
import java.util.ArrayList;
import java.util.Date;
```

Unused import statements are removed. This is an extremely convenient feature, and it is a good idea to find out how to accomplish it with your IDE.

---

Most of the time, you just import the packages that you need, without worrying too much about them. The only time that you need to pay attention to packages is when you have a name conflict. For example, both the `java.util` and `java.sql` packages have a `Date` class. Suppose you write a program that imports both packages.

```
import java.util.*;
import java.sql.*;
```

If you now use the `Date` class, you get a compile-time error:

```
Date today; // ERROR--java.util.Date or java.sql.Date?
```

The compiler cannot figure out which `Date` class you want. You can solve this problem by adding a specific `import` statement:

```
import java.util.*;
import java.sql.*;
```

```
import java.util.Date;
```

What if you really need both `Date` classes? Then use the full package name with every class name:

```
var startTime = new java.util.Date();
var today = new java.sql.Date(. . .);
```

Locating classes in packages is an activity of the *compiler*. The bytecodes in class files always use full package names to refer to other classes.

---



**Note:** It bears repeating that the *only* benefit of the `import` statement is convenience. Imports allow you to use a *simple* name such as `LocalDate` instead of the *qualified* name `java.time.LocalDate`.

---



**Note:** C and C++ programmers sometimes confuse `import` with `#include`. The two have nothing in common. In C/C++, you must use `#include` to include the declarations of external features because the compiler does not look inside any source files except the one that it is compiling. The Java compiler will happily consult other source or class files.

In C++, the construction analogous to the package mechanism is the namespace feature. Think of the `package` and `import` statements in Java as the analogs of the `namespace` and `using` directives in C++.

---

### 4.8.4. Module Imports

As you will see in **Chapter 12**, Java packages can be organized into *modules*. As of Java 25, you can import all packages in a module with a directive such as the following:

```
import module java.xml;
```

The `java.xml` module is specialized for processing XML files. With the module import, you can enjoy the convenience of importing all of the packages used for that purpose.

For general programming tasks, the most interesting module is `java.base`, which includes the packages `java.lang`, `java.util`, `java.io`, and over thirty others.

You can import it with the directive:

```
import module java.base;
```

Since the `java.base` module includes the `java.time` and `java.util` packages, you can use unqualified names for classes in those packages, such as `LocalDate` and `Date`.

In a compact source file, the `java.base` module is imported automatically. If you only use the packages in that module, you don't need any `import` statements.

---



**Caution:** When you import a module, it is possible that two or more packages have a class with the same name. For example, the `java.base` module has classes `java.net.Proxy` and `java.lang.reflect.Proxy`. If you need one of those classes, you can import it, or use the qualified name.

---

### 4.8.5. Static Imports

A form of the `import` statement permits the importing of static methods and fields, not just classes.

For example, if you add the directive

```
import static java.lang.System.*;
```

to the top of your source file, then you can use the static methods and fields of the `System` class without the class name prefix:

```
err.println("Goodbye, World!"); // i.e., System.err
exit(0); // i.e., System.exit
```

You can also import a specific method or field:

```
import static java.lang.System.exit;
```

In practice, it seems doubtful that many programmers will want to abbreviate `System.err` or `System.exit`. The resulting code seems less clear. On the other hand,

```
sqrt(pow(x, 2) + pow(y, 2))
```

seems much clearer than

```
Math.sqrt(Math.pow(x, 2) + Math.pow(y, 2))
```

You can import enumerated constants:

```
import java.time.DayOfWeek;
import static java.time.DayOfWeek.*;
DayOfWeek w = FRIDAY; // Same as DayOfWeek.FRIDAY
```

### 4.8.6. Addition of a Class into a Package

To place classes inside a package, put the name of the package at the top of your source file, *before* the code that defines the classes in the package. For example, the file `Employee.java` in **Listing 4.9** starts out like this:

```
package com.horstmann.corejava;

public class Employee {
    . . .
}
```

If you don't put a `package` statement in the source file, then the classes in that source file belong to the *unnamed package*. The unnamed package has no package name. Up to now, all our example classes have been located in the unnamed package.



**Note:** The implicitly declared class of a compact compilation unit (with methods declared outside a class) is always in the unnamed package.

When using named packages, place source files into a subdirectory that matches the full package name. For example, all source files in the `com.horstmann.corejava` package should be in a subdirectory `com/horstmann/corejava` (`com\horstmann\corejava` on Windows).

Here is a sample program that is distributed over two packages: The `PackageDemo` class belongs to the unnamed package—see **Listing 4.8**. The `Employee` class, shown in **Listing 4.9**, belongs to the `com.horstmann.corejava` package. Therefore, the `Employee.java` file must be in a subdirectory `com/horstmann/corejava`. In other words, the directory structure is as follows:

```
. (base directory)
 ├─ PackageDemo.java
 └─ com/
    └─ horstmann/
       └─ corejava/
          └─ Employee.java
```

To run this program, simply change to the base directory and run the command

```
java PackageDemo.java
```

The compiler automatically finds the file `com/horstmann/corejava/Employee.java` and compiles it.

**Listing 4.8 PackageDemo.java**

```
1  import com.horstmann.corejava.Employee;
2  // the Employee class is defined in that package
3
4  /**
5   * This program demonstrates the use of packages.
6   */
7  void main() {
8      // because of the import statement, we don't have to use
9      // com.horstmann.corejava.Employee here
10     var harry = new Employee("Harry Hacker", 50000, 1989, 10, 1);
11
12     harry.raiseSalary(5);
13
14     // because of the static import statement, we don't have to use IO.printl
15     IO.println("name=" + harry.getName() + ",salary=" + harry.getSalary());
16 }
```

**Listing 4.9 com/horstmann/corejava/Employee.java**

```
1  package com.horstmann.corejava;
2  // the classes in this file are part of this package
3
4  import module java.base;
5  // import statements come after the package statement
6
7  /**
8   * This class is almost identical to the initial Employee class, but it is in
9   * a package. Note that the class and its methods are public.
10  */
```

```
11 public class Employee {
12     private String name;
13     private double salary;
14     private LocalDate hireDay;
15
16     public Employee(String name, double salary, int year, int month, int day)
17         this.name = name;
18         this.salary = salary;
19         hireDay = LocalDate.of(year, month, day);
20     }
21
22     public String getName() {
23         return name;
24     }
25
26     public double getSalary() {
27         return salary;
28     }
29
30     public LocalDate getHireDay() {
31         return hireDay;
32     }
33
34     public void raiseSalary(double byPercent) {
35         double raise = salary * byPercent / 100;
36         salary += raise;
37     }
38 }
```

### 4.8.7. Compiling with Packages

Except for demonstration programs, it is not common to use the unnamed package. Instead, classes would be distributed over several packages, such as `com.horstmann.corejava` and `com.mycompany`.

```
. (base directory)
└─ com/
   ├─ horstmann/
   │  └─ corejava/
   │     └─ Employee.java
   └─ mycompany/
      └─ MyProgram.java
```

You start the program from the *base* directory—that is, the directory containing the `com` directory:

```
java com/mycompany/MyProgram.java
```

So far, we have launched source files without compiling them. That is fine for simple demos, but for larger programs, it is better to compile the code first. The compiler produces a *class file* for each class, containing the virtual machine code of its methods. For example, after you call

```
javac com/mycompany/MyProgram.java
```

the subdirectories contain the class files:

```
. (base directory)
 └─ com/
    ├─ horstmann/
    │  └─ corejava/
    │     ├─ Employee.java
    │     └─ Employee.class
    └─ mycompany/
       ├─ MyProgram.java
       └─ MyProgram.class
```



**Note:** When you use an IDE or a build tool, the class files are typically placed in a separate directory tree.

To run the compiled program, you call

```
java com.mycompany.MyProgram
```

Note that the `javac` compiler operates on *files* (with file separators and an extension `.java`), whereas the Java virtual machine loads a *class* (with dot separators).



**Caution:** The compiler does *not* check the directory structure when it compiles source files. For example, suppose you have a source file that starts with the directive

```
package com.mycompany;
```

You can compile the file even if it is not contained in a subdirectory `com/mycompany`. The source file will compile without errors *if it doesn't*

*depend on other packages*. However, the resulting program will not run unless you first move all class files to the right place. The *virtual machine* won't find the classes if the packages don't match the directories.

### 4.8.8. Package Access

You have already encountered the access modifiers `public` and `private`. Features tagged as `public` can be used by any class. Private features can be used only by the class that defines them. If you don't specify either `public` or `private`, the feature (that is, the class, method, or variable) can be accessed by all methods in the same *package*.

Consider the program in **Listing 4.2**. The `Employee` class was not defined as a public class. Therefore, only the other classes (such as `EmployeeDemo`) in the same package—the unnamed package in this case—can access it. For classes, this is a reasonable default. However, for variables, this was an unfortunate choice. Variables must explicitly be marked `private`, or they will default to having package access. This, of course, breaks encapsulation. The problem is that it is awfully easy to forget to type the `private` keyword. Here is an example from the `Window` class in the `java.awt` package, which is part of the source code supplied with the JDK:

```
public class Window extends Container {
    String warningString;
    . . .
}
```

Note that the `warningString` variable is not `private`! That means the methods of all classes in the `java.awt` package can access this variable and set it to whatever they like (such as `"Trust me!"`). Actually, the only methods that access this variable are in the `Window` class, so it would have been entirely appropriate to make the variable private. Perhaps the programmer typed the code in a hurry and simply forgot the `private` modifier? Perhaps nobody cared? After more than twenty years, that variable is still not private. Not only that—new fields have been added to the class over time, and about half of them aren't private either.

This can be a problem. By default, packages are not closed entities. That is, anyone can add more classes to a package. Of course, hostile or clueless programmers can then add code that modifies variables with package access. For example, in early versions of Java, it was an easy matter to smuggle another class into the `java.awt` package. Simply start out the class with

```
package java.awt;
```

Then, place the resulting class file inside a subdirectory `java/awt`, and you have gained access to the internals of the `java.awt` package. Through this subterfuge, it was possible to modify warning strings (see **Figure 4.9**).




**Figure 4.9:** Changing the warning string in a window

Starting with version 1.2, the JDK implementors rigged the class loader to explicitly disallow loading of user-defined classes whose package name starts with `"java."`. Of course, your own classes don't benefit from that protection. Another mechanism, now obsolete, lets a JAR file declare packages as *sealed*, preventing third parties from augmenting them. Nowadays, you should use modules to encapsulate packages. Modules are discussed in detail in **Chapter 12**.

### 4.8.9. The Class Path

As you have seen, classes are stored in subdirectories of the file system. The path to the class must match the package name.

Class files can also be stored in a JAR (Java archive) file. A JAR file contains multiple class files and subdirectories in a compressed format, saving space and improving performance. When you use a third-party library in your programs, you will usually be given one or more JAR files to include. You will see in **Section 4.9.1** how to create your own JAR files.

---



**Tip:** JAR files use the ZIP format to organize files and subdirectories. You can use any ZIP utility to peek inside JAR files.

---

To share classes among programs, you need to do the following:

1. Place your class files inside a directory—for example, `/home/user/classdir`. Note that this directory is the *base* directory for the package tree. If you add the class `com.horstmann.corejava.Employee`,

then the `Employee.class` file must be located in the subdirectory `/home/user/classdir/com/horstmann/corejava`.

2. Place any JAR files inside a directory—for example, `/home/user/archives`.
3. Set the *class path*. The class path is the collection of all locations that can contain class files.

In UNIX, the elements on the class path are separated by colons:

```
/home/user/classdir:.:/home/user/archives/archive.jar
```

In Windows, they are separated by semicolons:

```
c:\classdir;.;c:\archives\archive.jar
```

In both cases, the period denotes the current directory.

This class path contains

- The base directory `/home/user/classdir` or `c:\classdir`;
- The current directory (`.`); and
- The JAR file `/home/user/archives/archive.jar` or `c:\archives\archive.jar`.

You can specify a wildcard for a JAR file directory, like this:

```
/home/user/classdir:.:/home/user/archives/'*'
```

or

```
c:\classdir;.;c:\archives\*
```

In UNIX, the `*` must be escaped to prevent shell expansion.

All JAR files (but not `.class` files) in the `archives` directory are included in this class path.

The Java API is always searched for classes; don't include it explicitly in the class path.

---

**Caution:** The `javac` compiler always looks for files in the current directory, but the `java` virtual machine launcher only looks into the current direc-

tory if the “ . ” directory is on the class path. If you have no class path set, it’s not a problem—the default class path consists of the “ . ” directory. But if you have set the class path and forgot to include the “ . ” directory, your programs will compile without error, but they won’t run.

---

The class path lists all directories and archive files that are *starting points* for locating classes. Let’s consider our sample class path:

```
/home/user/classdir:.:/home/user/archives/archive.jar
```

Suppose the virtual machine searches for the class file of the `com.horstmann.corejava.Employee` class. It first looks in the Java API classes. It won’t find the class file there, so it turns to the class path. It then looks for the following files:

- `/home/user/classdir/com/horstmann/corejava/Employee.class`
- `com/horstmann/corejava/Employee.class` starting from the current directory
- `com/horstmann/corejava/Employee.class` inside `/home/user/archives/archive.jar`

The compiler has a harder time locating files than does the virtual machine. If you refer to a class without specifying its package, the compiler first needs to find out the package that contains the class. It consults all `import` directives as possible sources for the class. For example, suppose the source file contains directives

```
import java.util.*;
import com.horstmann.corejava.*;
```

and the source code refers to a class `Employee`. The compiler then tries to find `java.lang.Employee` (because the `java.lang` package is always imported by default), `java.util.Employee`, `com.horstmann.corejava.Employee`, and `Employee` in the current package. It searches for *each* of these classes in all of the locations of the class path. It is a compile-time error if more than one class is found. (Qualified names must be unique, so the order of the `import` statements doesn’t matter.)

The compiler goes one step further. It looks at the *source files* to see if the source is newer than the class file. If so, the source file is recompiled automatically. Recall that you can import only public classes from other packages. A source file can only contain one public class, and the names of the file and the public class must match. Therefore, the compiler can easily locate source files for public

classes. However, you can import nonpublic classes from the current package. These classes may be defined in source files with different names. If you import a class from the current package, the compiler searches *all* source files of the current package to see which one defines the class.

### 4.8.10. Setting the Class Path

IDEs and build tools have their own mechanisms for managing the class path. But it is a good idea to know how to set it by hand so you can troubleshoot any configuration issues.

It is best to specify the class path with the option `-classpath` (or `-cp` or the more modern variant, `--class-path`):

```
java -classpath /home/user/classdir:.:/home/user/archives/archive.jar MyProg
```

or

```
java -classpath c:\classdir;.;c:\archives\archive.jar MyProg
```

The entire command must be typed onto a single line. It is a good idea to place such a long command line into a shell script or a batch file.

Using the `-classpath` option is the preferred approach for setting the class path. An alternate approach is the `CLASSPATH` environment variable. The details depend on your shell. With the Bourne Again shell (bash), use the command

```
export CLASSPATH=/home/user/classdir:.:/home/user/archives/archive.jar
```

With the Windows shell, use

```
set CLASSPATH=c:\classdir;.;c:\archives\archive.jar
```

The class path is set until the shell exits.

---



**Note:** When using a JAR in `jshell`, launch it with

```
CLASSPATH=archive.jar jshell
```

The seemingly equivalent

```
jshell --class-path=archive.jar
```

doesn't give you autocomplete for the classes in the JAR file (see https://bugs.openjdk.org/browse/JDK-8177650).



**Caution:** Some people recommend to set the `CLASSPATH` environment variable permanently. This is generally a bad idea. People forget the global setting and are surprised when their classes are not loaded properly. A particularly reprehensible example was Apple's QuickTime installer in Windows. For several years, it globally set `CLASSPATH` to point to a JAR file it needed, but did not include the current directory in the classpath. As a result, countless Java programmers were driven to distraction when their programs compiled but failed to run.



**Caution:** In the past, some people recommended to bypass the class path altogether, by dropping all JAR files into the `jre/lib/ext` directory. That mechanism is obsolete with Java 9, but it was always bad advice. It was easy to get confused when long-forgotten classes were loaded from the extension directory.



**Note:** Classes can also be loaded from the *module path*. Modules and the module path are discussed in Chapter 12.

## 4.9. JAR Files

When you package your application, you want to give your users a single file, not a directory structure filled with class files. Java Archive (JAR) files were designed for this purpose. A JAR file can contain both class files and other file types such as image and sound files. Moreover, JAR files are compressed, using the familiar ZIP compression format.

### 4.9.1. Creating JAR files

Use the `jar` tool to make JAR files. (In the default JDK installation, it's in the `$JAVA_HOME/bin` directory.) The most common command to make a new JAR file uses the following syntax:

```
jar cvf jarFileName file1 file2 . . .
```

For example:

```
jar cvf CalculatorClasses.jar *.class icon.png
```

In general, the `jar` command has the following format:

```
jar options file1 file2 . . .
```

**Table 4.2** lists all the options for the `jar` program. They are similar to the options of the UNIX `tar` command.

**Table 4.2:** `jar` Program Options

| Option | Description |
|---|---|
| `c` | Creates a new or empty archive and adds files to it. If any of the specified file names are directories, the `jar` program processes them recursively. |
| `C` | Temporarily changes the directory. For example,<br>`jar cvf jarFileName.jar -C classes *.class`<br>changes to the `classes` subdirectory to add class files. |
| `e` | Creates an entry point in the manifest (see **Section 4.9.3**). |
| `f` | Specifies the JAR file name as the second command-line argument. If this argument is missing, `jar` will write the result to standard output (when creating a JAR file) or read it from standard input (when extracting or tabulating a JAR file). |
| `i` | Creates an index file (for speeding up lookups in a large archive). |
| `m` | Adds a *manifest* to the JAR file. A manifest is a description of the archive contents and origin. Every archive has a default manifest, but you can supply your own if you want to authenticate the contents of the archive. |
| `M` | Does not create a manifest file for the entries. |
| `t` | Displays the table of contents. |

| Option | Description |
|---|---|
| `u` | Updates an existing JAR file. |
| `v` | Generates verbose output. |
| `x` | Extracts files. If you supply one or more file names, only those files are extracted. Otherwise, all files are extracted. |
| `0` | Stores without ZIP compression. |

You can package application programs and code libraries into JAR files. For example, if you want to send mail in a Java program, you use a library that is packaged in a file `javax.mail.jar`.

### 4.9.2. The Manifest

In addition to class files, images, and other resources, each JAR file contains a *manifest* file that describes special features of the archive.

The manifest file is called `MANIFEST.MF` and is located in a special `META-INF` subdirectory of the JAR file. The minimum legal manifest is quite boring—just

```
Manifest-Version: 1.0
```

Complex manifests can have many more entries. The manifest entries are grouped into sections. The first section in the manifest is called the *main section*. It applies to the whole JAR file. Subsequent entries can specify properties of named entities such as individual files, packages, or URLs. Those entries must begin with a `Name` entry. Sections are separated by blank lines. For example:

```
Manifest-Version: 1.0
lines describing this archive

Name: Woozle.class
lines describing this file
Name: com/mycompany/mypkg/
lines describing this package
```

To edit the manifest, place the lines that you want to add to the manifest into a text file. Then run

```
jar cfm jarFileName manifestFileName . . .
```

For example, to make a new JAR file with a manifest, run

```
jar cfm MyArchive.jar manifest.mf com/mycompany/mypkg/*.class
```

To update the manifest of an existing JAR file, place the additions into a text file and use a command such as

```
jar ufm MyArchive.jar manifest-additions.mf
```



**Note:** See https://docs.oracle.com/en/java/javase/25/docs/specs/jar/jar.html for more information on the JAR and manifest file formats.

### 4.9.3. Executable JAR Files

You can use the `e` option of the `jar` command to specify the *entry point* of your program—the class that you would normally specify when invoking the `java` program launcher:

```
jar cvfe MyProgram.jar com.mycompany.mypkg.MainAppClass files to add
```

Alternatively, you can specify the *main class* of your program in the manifest, including a statement of the form

```
Main-Class: com.mycompany.mypkg.MainAppClass
```

Do not add a `.class` extension to the main class name.



**Caution:** The last line in the manifest must end with a newline character. Otherwise, the manifest will not be read correctly. It is a common error to produce a text file containing just the `Main-Class` line without a line terminator.

With either method, users can simply start the program as

```
java -jar MyProgram.jar
```

Depending on the operating system configuration, users may even be able to launch the application by double-clicking the JAR file icon. Here are behaviors for various operating systems:

- On Windows, the Java runtime installer creates a file association for the "`.jar`" extension that launches the file with the `javaw -jar` command. (Unlike the `java` command, the `javaw` command doesn't open a shell window.)
- On Mac OS X, the operating system recognizes the "`.jar`" file extension and executes the Java program when you double-click a JAR file.

However, a Java program in a JAR file does not have the same feel as a native application. On Windows, you can use third-party wrapper utilities that turn JAR files into Windows executables. A wrapper is a Windows program with the familiar `.exe` extension that locates and launches the Java virtual machine (JVM) or tells the user what to do when no JVM is found. There are a number of commercial and open-source products, such as Launch4J (**https://launch4j.sourceforge.net**) and IzPack (**https://github.com/izpack/izpack**).

### 4.9.4. Multi-Release JAR Files

With the introduction of modules and strong encapsulation of packages, some previously accessible internal APIs are no longer available. This may require library providers to distribute different code for different Java versions. *Multi-release JARs* take care of this use case.

For backward compatibility, version-specific class files are placed in the `META-INF/versions` directory:

```
Application.class
BuildingBlocks.class
Util.class
META-INF
 ├─ MANIFEST.MF (with line Multi-Release: true)
 └─ versions
    ├─ 9
    │  ├─ Application.class
    │  └─ BuildingBlocks.class
    └─ 10
       └─ BuildingBlocks.class
```

Suppose the `Application` class makes use of the `CssParser` class. Then the legacy `Application.class` file can be compiled to use

`com.sun.javafx.css.CssParser`, while the Java 9 version uses `javafx.css.CssParser`.

Java 8 knows nothing about the `META-INF/versions` directory and will simply load the legacy classes. When the JAR file is read by Java 9, the new version is used instead.

To add versioned class files, use the `--release` flag:

```
jar uf MyProgram.jar --release 9 Application.class
```

To build a multi-release JAR file from scratch, use the `-C` option and switch to a different class file directory for each version:

```
jar cf MyProgram.jar -C bin/8 . --release 9 -C bin/9 Application.class
```

When compiling for different releases, use the `--release` flag and the `-d` flag to specify the output directory:

```
javac -d bin/8 --release 8 . . .
```

The `-d` option creates the directory if it doesn't exist.

The JDK ships with symbol files for two prior versions of the API. In Java 21, you can compile with `--release` set to `21`, `20`, or `19`.

Multi-release JARs are not intended for different versions of a program or library. The public API of all classes should be the same for both releases. The sole purpose of multi-release JARs is to enable a particular version of your program or library to work with multiple JDK releases. If you add functionality or change an API, you should provide a new version of the JAR instead.

---



**Note:** Tools such as `javap` are not retrofitted to handle multi-release JAR files. If you call

```
javap -classpath MyProgram.jar Application.class
```

you get the base version of the class (which, after all, is supposed to have the same public API as the newer version). If you must look at the newer version, call

```
javap -classpath MyProgram.jar\!/META-INF/versions/9/Application.class
```

### 4.9.5. A Note about Command-Line Options

The options of commands in the Java Development Kit have traditionally used single dashes followed by multiletter option names, such as

```
java -jar . . .
javac -Xlint:fallthrough -classpath . . .
```

The exception was the `jar` command, which followed the classic option format of the `tar` command without dashes:

```
jar cvf . . .
```

JEP 293 (**https://openjdk.org/jeps/293**) provides guidelines for moving toward a more common option format, starting with Java 9. With most Linux tools, multiletter option names are preceded by double dashes, with single-letter shortcuts for common options. For example, the Linux `ls` command can be called with a "human-readable" option as

```
ls --human-readable
```

or

```
ls -h
```

With `javac` and `java`, you can now use `--version` instead of `-version` and `--class-path` instead of `-classpath`. As you will see in **Chapter 12**, the `--module-path` option has a shortcut `-p`.

Arguments of options with `--` and multiple letters are separated by whitespace or an `=` sign:

```
java --class-path /home/user/classdir . . .
```

or

```
java --class-path=/home/user/classdir . . .
```

The `=` form is useful when an option needs to be passed through to another tool, where the passthrough mechanism cannot handle white space.

---



**Tip:** You can use the `JDK_JAVA_OPTIONS` environment variable to pass command-line options to the `java` launcher:

```
export JDK_JAVA_OPTIONS='--class-path /home/user/classdir -enableassertions'
```

---



**Caution:** According to JEP 293, single-letter options without arguments can be grouped together:

```
jar -c -v -f myapp.jar -m MANIFEST.MF *.class
```

can be simplified to

```
jar -cv -f myapp.jar -m MANIFEST.MF *.class
```

But that does not currently work. Strangely,

```
jar -cvfm myapp.jar MANIFEST.MF *.class
```

works even though it should not. Until this is sorted out, it seems best to stick with the classic `tar`-style options for the `jar` command.

---



**Caution:** According to JEP 293, arguments of single-letter options can be separated by whitespace or directly follow the option:

```
javac -d outputdir . . .
```

or

```
javac -doutputdir . . .
```

The latter doesn't currently work. It also seems like a bad idea since it would invite conflicts with legacy options. For example, what should happen if the output directory happens to be named `eprecation`?

---

## 4.10. Documentation Comments

The JDK contains a very useful tool, called `javadoc`, that generates HTML documentation from your source files. In fact, the online API documentation described in **Chapter 3** is simply the result of running `javadoc` on the source code of the standard Java library.

If you add comments that start with the special delimiter `/**` to your source code, you too can easily produce professional-looking documentation. This is a very nice approach because it lets you keep your code and documentation in one place. If you put your documentation into a separate file, then, as you probably know, the code and comments tend to diverge over time. When documentation comments are in the same file as the source code, it is an easy matter to update both and run `javadoc` again.

### 4.10.1. Comment Insertion

The `javadoc` utility extracts information for the following items:

- Modules
- Packages
- Public classes and interfaces
- Public and protected fields
- Public and protected constructors and methods

Protected members are introduced in **Chapter 5**, interfaces in **Chapter 6**, and modules in **Chapter 12**.

You can (and should) supply a comment for each of these members. Each comment is placed immediately *above* the member it describes. A comment starts with a `/**` and ends with a `*/`.

Each `/** . . . */` documentation comment contains *free-form text* followed by *tags*. A tag starts with an `@`, such as `@since` or `@param`.

The *first sentence* of the free-form text should be a *summary statement*. The `javadoc` utility automatically generates summary pages that extract these sentences.

The most common `javadoc` tags are *block tags*. They must appear at the beginning of a line and start with `@`, optionally preceded by whitespace, the comment delimiter `/**`, or leading `*` which are often used for multiline comments. In contrast, *inline tags* are enclosed in braces: `{@tagname contents}`. The contents may contain braces, but they must be balanced. Examples are the `@code` and `@link` tags.

JavaDoc text can contain HTML tags such as `<em>` or `ul`. They are passed through to the generated HTML pages.

---



**Note:** Starting with Java 23, you can author JavaDoc comments in Markdown instead of plain text and HTML. This can be easier to read and write, particularly with complex mixtures of text and code.

Markdown comments are delimited with `///` in front of each line, instead of `/**` and `*/`. This makes it easy to include code with comments.

JavaDoc follows the CommonMarc specification, and in addition supports the Github Flavored Markdown extension for tables.

At this point, JavaDoc Markdown comments are not yet commonly used. You can find the details at **https://openjdk.org/jeps/467**.

---

### 4.10.2. Class Comments

The class comment must be placed *after* any `import` statements, directly before the `class` definition.

Here is an example of a class comment:

```
/**
 * A {@code Card} object represents a playing card, such
 * as "Queen of Hearts". A card has a suit (Diamond, Heart,
 * Spade or Club) and a value (1 = Ace, 2 . . . 10, 11 = Jack,
 * 12 = Queen, 13 = King).
 */
public class Card {
    . . .
}
```

---



**Note:** There is no need to add an `*` in front of every line. For example, the following comment is equally valid:

```
/**
    A <code>Card</code> object represents a playing card, such
    as "Queen of Hearts". A card has a suit (Diamond, Heart,
    Spade or Club) and a value (1 = Ace, 2 . . . 10, 11 = Jack,
    12 = Queen, 13 = King).
*/
```

However, most IDEs supply the asterisks automatically and rearrange them when the line breaks change.

### 4.10.3. Method Comments

Each method comment must immediately precede the method that it describes. In addition to the general-purpose tags, you can use the following tags:

- `@param` variable description
  This tag adds an entry to the "parameters" section of the current method. The description can span multiple lines and can use HTML tags. All `@param` tags for one method must be kept together.
- `@return` description
  This tag adds a "returns" section to the current method. The description can span multiple lines and can use HTML tags.
- `@throws` class description
  This tag adds a note that this method may throw an exception. Exceptions are the topic of **Chapter 7**.

Here is an example of a method comment:

```
/**
 * Raises the salary of an employee.
 * @param  byPercent the percentage by which to raise the salary (e.g., 10 means 10
 * @return  the amount of the raise
 */
public double raiseSalary(double byPercent) {
    double raise = salary * byPercent / 100;
    salary += raise;
    return raise;
}
```



**Tip:** It can be tedious to write comments for methods whose description and return value are identical, such as:

```
/**
 * Returns the name of the employee.
 * @return  the name of the employee
 */
```

In such cases, consider the inline form of `@return` introduced in Java 16:

```
/**
 * {@return the name of the employee}
 */
```

The description section becomes "Returns the name of the employee.", and a "Returns" section with the same contents is added.



**Note:** Methods can also have a `@throws` tag to document the exceptions that a method throws. Exceptions are covered in **Chapter 7**.



**Note:** If you add `{@inheritDoc}` into a method description or the `@param`, `@return`, or `@throws` tag bodies, then the documentation from the method in a superclass or interface is copied verbatim. (See **Chapter 5** and **Chapter 6** for superclasses and interfaces.)

### 4.10.4. Field Comments

You only need to document public fields—generally that means static constants. For example:

```
/**
 * The "Hearts" card suit
 */
public static final int HEARTS = 1;
```

### 4.10.5. Package Comments

Place the class, method, and variable comments directly into the Java source files, delimited by `/** . . . */` documentation comments. However, to generate *package* comments, you need to add a separate file in each package directory. You have two choices:

1. Supply a Java file named `package-info.java`. The file must contain an initial documentation comment, delimited with `/**` and `*/`, followed by a `package` statement. It should contain no further code or comments.

2. Supply an HTML file named `package.html`. All text between the tags `<body>. . .</body>` is extracted.

### 4.10.6. HTML Markup

In the free-form text, you can use HTML modifiers such as `<em>. . .</em>` for emphasis, `<strong>. . .</strong>` for strong emphasis, `<ul>`/`<li>` for bulleted lists, and `<img . . ./>` to include an image.

To type monospaced code, use `{@code . . . }` instead of `<code>. . .</code>`—then you don't have to worry about escaping `<` and `&` characters inside the code. If you want to write unescaped `<` or `&` in the plain font, use the `@literal{ . . .}` tag.

For multiline code displays in an HTML `pre` tag, you can use:

```
/**
 * . . .
 * <pre>{@code
 *      . . .
 *      . . .
 * }</pre>
 */
```

or, since Java 18:

```
/**
 * . . .
 * {@snippet :
 *      . . .
 *      . . .
 * }
 */
```

Either way, you don't have to escape `<` or `&` characters in the code. However, braces have to match, and you cannot have `/* . . . */` comments.

### 4.10.7. Links

You can use hyperlinks to other relevant parts of the `javadoc` documentation, or to external documents, with the `@see` and `@link` tags.

The tag `@see` reference adds a hyperlink in the "see also" section. It can be used with both classes and methods. Here, reference can be one of the following:

```
package.class#member  label
<a href=". . .">label</a>
"text"
```

The first case is the most useful. You supply the name of a class, method, or variable, and `javadoc` inserts a hyperlink to the documentation. For example,

```
@see com.horstmann.corejava.Employee#raiseSalary(double)
```

makes a link to the `raiseSalary(double)` method in the `com.horstmann.corejava.Employee` class. You can omit the name of the package, or both the package and class names. Then, the member will be located in the current package or class.

Note that you must use a `#`, not a period, to separate the class from the method or variable name. The Java compiler itself is highly skilled in determining the various meanings of the period character as separator between packages, subpackages, classes, inner classes, and methods and variables. But the `javadoc` utility isn't quite as clever, so you have to help it along.

Constructors have the special name `<init>`, not the name of the class, such as

```
@see com.horstmann.corejava.Employee#<init>()
```

You can specify an optional label after the member that will appear as the link anchor. If you omit the label, the user will see the member name.

If the `@see` tag is followed by a `<` character, then you need to specify a hyperlink. You can link to any URL you like. For example:

```
@see <a href="horstmann.com/corejava.html">The Core Java home page</a>
```

If the `@see` tag is followed by a `"` character, then the text is displayed in the "see also" section. For example:

```
@see "Core Java Volume 2"
```

You can add multiple `@see` tags for one member, but you must keep them all together.

If you like, you can place hyperlinks to other classes or methods anywhere in any of your documentation comments. Insert a tag of the form

```
{@link package.class#member}
```

anywhere in a comment. The member reference follows the same rules as for the `@see` tag.

In a code snippet, place the `@link` tag in the comment, so that it doesn't interfere with the code:

```
{@snippet
    IO.println(); // @link substring=println target=java.lang.IO#println()
}
```

Since Java 20, ids are automatically generated for level 2 and level 3 headings. For example,

```
<h2>General Principles</h2>
```

gets an id `general-principles-heading`, which you can refer from `@see` and `@link` tags. You need two `#` symbols to link to an id:

```
{@link com.horstmann.corejava.Employee##general-principles-heading}
```

Use `@linkplain` instead of `@link` if a link should be displayed in the plain font instead of the code font.



**Note:** If your comments contain links to other files, such as images (for example, diagrams or images of user interface components), place those files into a subdirectory, named `doc-files`, of the directory containing the source file. The `javadoc` utility will copy the `doc-files` directories and their contents from the source directory to the documentation directory. You need to use the `doc-files` directory in your link, for example `<img src="doc-files/uml.png" alt="UML diagram"/>`.

### 4.10.8. General Comments

The tag `@since text` makes a "since" entry. The `text` can be any description of the version that introduced this feature. For example, `@since 1.7.1`.

The following tags can be used in class documentation comments:

- `@author name`
  This tag makes an "author" entry. You can have multiple `@author` tags, one for each author. Don't feel compelled to use this tag—your version control system does a more thorough job tracking authorship.
- `@version text`
  This tag makes a "version" entry. The `text` can be any description of the current version.

The `{@value optionalFormat constantFieldReference}` inline tag inserts the value of a constant field, with an optional `printf`-style formatter since Java 20. For example, `{@value %X Integer#MAX_VALUE}` yields 7FFFFFFF.

The `@deprecated` tag is used together with the `@Deprecated` annotation. It is followed by text that tells the user when and why the deprecation occurred, and to describe alternatives.

Finally, you can use the `{@index entry}` tag to add an entry to the generated index.

### 4.10.9. Code Snippets

The purpose of this feature is to allow you to import (presumably well-tested and up-to-date) code from external files. This minimizes the risk of incorrect or stale documentation. Code snippets were introduced in Java 18.

You can import an entire file with these tags:

```
{@snippet file=EmployeeDemo.java}
{@snippet class=com.horstmann.corejava.EmployeeDemo.java}
```

Files should be placed in the `snippet-files` subdirectory of the current package.

More commonly, you want to copy a *region* from a file. In the source file, you specify the name and extent of a region as follows:

```
public class EmployeeDemo {
    . . .
    // @start region=default-employee
    var e = new Employee();
    String name = e.getName(); // name is null
```

```
    // @end
    . . .
}
```

To highlight a part of the snippet, use a comment:

```
var e = new Employee(); // @hightlight substring=new
String name = e.getName(); // @highlight regex=get[A-Z][a-z]+
```

Inside a region, you can replace a part with another string, for example to make the documentation more general:

```
var e = new Employee("Fred", 100000); // @replace regex=([^)]+) replacement="(...,
```

Then the documentation contains:

```
var e = new Employee(..., ...);
```

To add a link, use:

```
var c = new Card(); // @link substring=Card target=com.horstmann.games#Card.<init>(
```

These decorations and transformations are specified as comments, so that your source file compiles and runs.

### 4.10.10. Comment Extraction

Here, `docDirectory` is the name of the directory where you want the HTML files to go. Follow these steps:

1. Change to the directory that contains the source files you want to document. If you have nested packages to document, such as `com.horstmann.corejava`, you must be working in the directory that contains the subdirectory `com`. (This is the directory that contains the `overview.html` file, if you supplied one.)
2. Run the command

   ```
   javadoc -d docDirectory nameOfPackage
   ```

   for a single package. Or, run

```
javadoc -d docDirectory nameOfPackage1 nameOfPackage2. . .
```

to document multiple packages. If your files are in the unnamed package, run instead

```
javadoc -d docDirectory *.java
```

If you omit the `-d docDirectory` option, the HTML files are extracted to the current directory. That can get messy, and I don't recommend it.

The `javadoc` program can be fine-tuned by numerous command-line options. For example, you can use the `-author` and `-version` options to include the `@author` and `@version` tags in the documentation. (By default, they are omitted.) Another useful option is `-link`, to include hyperlinks to standard classes. For example, if you use the command

```
javadoc -link https://docs.oracle.com/en/java/javase/25/docs/api *.java
```

all standard library classes are automatically linked to the documentation on the Oracle web site.

If you use the `-linksource` option, each source file is converted to HTML (without color coding, but with line numbers), and each class and method name turns into a hyperlink to the source.

You can also supply an overview comment for all source files. Place it in a file such as `overview.html` and run the `javadoc` tool with the command line option `-overview filename`. All text between the tags `<body>. . .</body>` is extracted. The content is displayed when the user selects "Overview" from the navigation bar.

For additional options, refer to the online documentation of the `javadoc` utility at **https://docs.oracle.com/en/java/javase/25/javadoc/index.html**.

## 4.11. Class Design Hints

Without trying to be comprehensive or tedious, I want to end this chapter with some hints that will make your classes more acceptable in well-mannered OOP circles.

1. *Always keep data private.*

This is first and foremost; doing anything else violates encapsulation. You may need to write an accessor or mutator method occasionally, but you are still better off keeping the instance fields private. Bitter experience shows that the data representation may change, but how these data are used will change much less frequently. When data are kept private, changes in their representation will not affect the users of the class, and bugs are easier to detect.

2. *Always initialize data.*
   Java won't initialize local variables for you, but it will initialize instance fields of objects. Don't rely on the defaults, but initialize all variables explicitly, either by supplying a default or by setting defaults in all constructors.

3. *Don't use too many basic types in a class.*
   The idea is to replace multiple *related* uses of basic types with other classes. This keeps your classes easier to understand and to change. For example, replace the following instance fields in a `Customer` class:

   ```
   private String street;
   private String city;
   private String state;
   private int zip;
   ```

   with a new class called `Address`. This way, you can easily cope with changes to addresses, such as the need to deal with international addresses.

4. *Not all fields need individual field accessors and mutators.*
   You may need to get and set an employee's salary. You certainly won't need to change the hiring date once the object is constructed. And, quite often, objects have instance fields that you don't want others to get or set, such as an array of state abbreviations in an `Address` class.

5. *Break up classes that have too many responsibilities.*
   This hint is, of course, vague: "too many" is obviously in the eye of the beholder. However, if there is an obvious way to break one complicated class into two classes that are conceptually simpler, seize the opportunity. (On the other hand, don't go overboard; ten classes, each with only one method, are usually an overkill.)
   Here is an example of a bad design:

   ```
   public class CardDeck { // bad design
       private int[] value;
       private int[] suit;

       public CardDeck() { . . . }
       public void shuffle() { . . . }
       public int getTopValue() { . . . }
   ```

```
    public int getTopSuit() { . . . }
    public void draw() { . . . }
}
```

This class really implements two separate concepts: a *deck of cards*, with its `shuffle` and `draw` methods, and a *card*, with the methods to inspect its value and suit. It makes sense to introduce a `Card` class that represents an individual card. Now you have two classes, each with its own responsibilities. Since the `Card` class is so simple, it can be implemented as a record. Also, instead of using integer values for the suits, an enumeration is a better choice:

```
public class CardDeck {
    private Card[] cards;

    public CardDeck() { . . . }
    public void shuffle() { . . . }
    public Card getTop() { . . . }
    public void draw() { . . . }
}

public enum Suit { DIAMONDS, HEARTS, SPADES, CLUBS }

public record Card(int value, Suit suit) { }
```

6. *Make the names of your classes and methods reflect their responsibilities.*
   Just as variables should have meaningful names that reflect what they represent, so should classes. (The standard library certainly contains some dubious examples, such as the `Date` class that describes time.)
   A good convention is that a class name should be a noun (`Order`), or a noun preceded by an adjective (`RushOrder`) or a gerund (an "-ing" word, as in `BillingAddress`). As for methods, follow the standard convention that accessor methods begin with a lowercase `get` (`getSalary`) and mutator methods use a lowercase `set` (`setSalary`).
7. *Prefer immutable classes.*
   The `LocalDate` class, and other classes from the `java.time` package, are immutable—no method can modify the state of an object. Instead of mutating objects, methods such as `plusDays` return new objects with the modified state.
   The problem with mutation is that it can happen concurrently when multiple threads try to update an object at the same time. The results are unpredictable. When classes are immutable, it is safe to share their objects among multiple threads.

Therefore, it is a good idea to make classes immutable when you can. This is particularly easy with classes that represent values, such as a string or a point in time. Computations can simply yield new values instead of updating existing ones.

Of course, not all classes should be immutable. It would be strange to have the `raiseSalary` method return a new `Employee` object when an employee gets a raise.

In this chapter, we covered the fundamentals of objects and classes that make Java an "object-based" language. In order to be truly object-oriented, a programming language must also support inheritance and polymorphism. The Java support for these features is the topic of the next chapter.

Previous chapter
< 3. Fundamental Programming Structures in Java

Next chapter
5. Inheritance >